

[두산로보틱스] 지능형 로보틱스 엔지니어

# 더듬이 로봇

협동-1: ROS2를 활용한 로봇 자동화 공정 시스템 구현 프로젝트

**TEAM 더듬이**

양준호, 윤재현, 임지현, 권도원, 정제유

**멘토**

손미란, 박윤현

# 목 차

---

01 프로젝트 개요

02 프로젝트 팀 구성 및 역할

03 프로젝트 수행 절차 및 방법

04 프로젝트 수행 경과

05 자체 평가 의견

# 01 프로젝트 개요 | 주제 및 배경

---

## 기존 비전 센서의 한계 극복

- ✓ **기존 방식의 문제:** 투명한 사출물은 빛 반사 및 굴절로 인해 카메라 기반 비전 시스템에서 높은 오탐률 발생
- ✓ **기획 의도:** 카메라 없이 로봇의 센서만 이용하여 물리적 스캔(접촉) 방식으로 결함을 감지하는 독자적 시스템 구축
- ✓ **차별화 포인트:** 비전 환경에 구애받지 않는 투명 재질 전용 자동 검수 솔루션



# 01 프로젝트 개요 | 프로젝트 내용

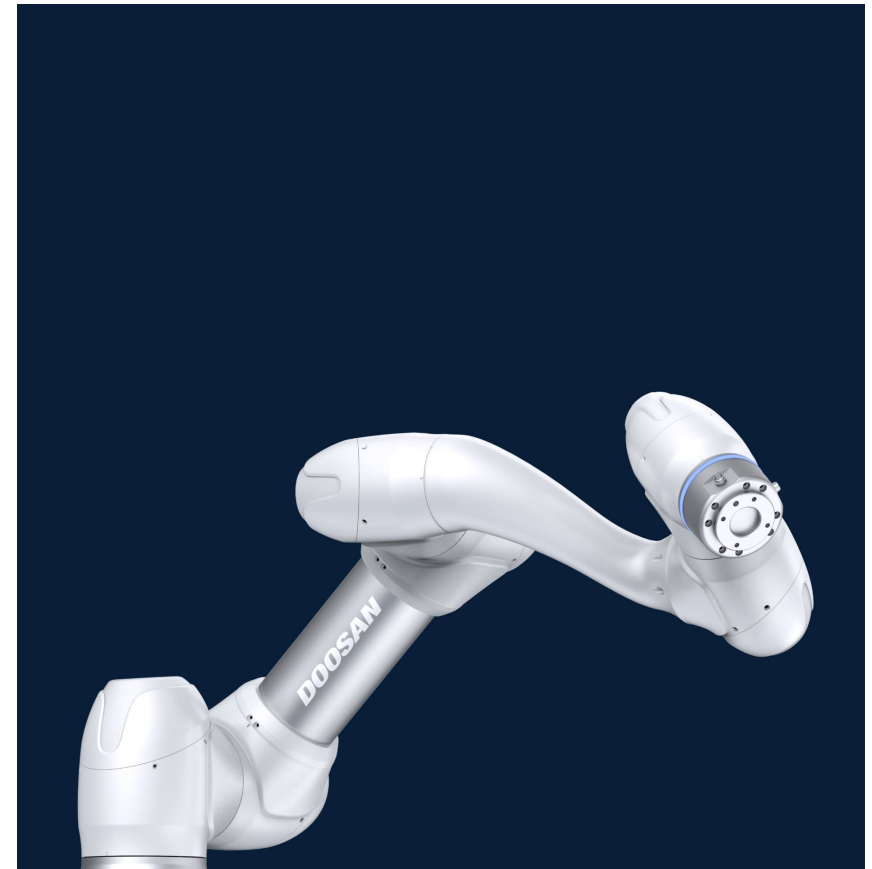
---

## ⚙️구현 내용 및 컨셉

컨베이어 벨트로 이동하는 투명 사출물을 협동로봇이 센서로 직접 표면을 굽으며(Scanning) 미세한 흠집과 결함을 실시간으로 감지하고 분류합니다.

## 🏠훈련 내용과의 연관성

교육 과정에서 습득한 두산 M0609 협동로봇 제어 기술을 기반으로, 외부 센서 데이터 피드백과 정밀 모션 제어를 융합하여 실제 산업 현장의 난제(투명물 검수)를 해결하는 실무형 프로젝트입니다.



# 01 프로젝트 개요 | 활용방안 및 기대 효과

## 자동화 공정의 마지막 퍼즐

본 시스템은 스마트 팩토리의 최종 검수 라인에서 비전 카메라의 한계로 인해 수동으로 이루어지던 **투명 사출물 검수 공정**을 100% 자동화할 수 있습니다.

- ✓ **품질 향상:** 카메라로 식별 불가능한 미세 표면 결함 감지
- ✓ **비용 절감:** 비싼 비전 시스템 및 전문 인력 대체
- ✓ **범용성:** 유리, 아크릴, 투명 플라스틱 등 다양한 재질 대응 가능



# 01 프로젝트 개요 | 시스템 구성 | HW · SW · DB

## Hardware

- Doosan m0609**  
6축 협동로봇 및 내장 토크 센서
- Arduino & HC-SR04**  
아두이노 및 초음파 센서 제어
- Conveyor**  
컨테이너 벨트
- Acrylic Hemisphere**  
아크릴 반구모형
- Workstation**  
RTDE 통신 및 데이터 처리 PC
- Custom Tool**  
3D 프린팅 자체제작 툴

## Software & Database

| 소통                                    | 개발 및 API                                                                                                                                                                                      | 통합 환경          | Database                                                                                    |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|---------------------------------------------------------------------------------------------|
| <b>Slack / Notion</b><br>팀 협업 및 일정 관리 | ROS2 (rclpy)<br>DSR_ROBOT2<br>dsr_msgs2<br>arduino_ros bridge<br>Firebase_rosbridge<br>JS<br><a href="#">Plotly.js</a><br>HTML/CSS<br>Python<br>Firebase SDK (Admin / Web)<br>OpenCV, PyTorch | <b>VS Code</b> | <b>Firebase</b><br>Realtime DB, Cloud Firestore<br><b>Firebase Storage</b><br>Cloud Storage |

실시간 데이터 핸들링 및 클라우드 DB 연동: 100Hz+ 확보 완료

## 02 프로젝트 팀 구성 및 역할

지현

임지현

- GUI 메인 개발
- 데이터 분석 보조
- 물체 검사 경로 보조
- 통합 테스트 (미니/큰)

제유

정제유

- 분류 시퀀스 메인
- 시작공정 로직 보조
- GUI 디자인 보조
- 시스템 통합 제어

호준

양준호

- 시작공정/컨베이어
- 3D 모델링 제작
- 클라우드 포인트 생성
- 임베디드 로직 설계
- 전체 일정 조율

도현

권도현

- 분석 엔진 메인
- 검사 경로 알고리즘
- 물체 감지 센서 최적화
- 전체 통합 테스트

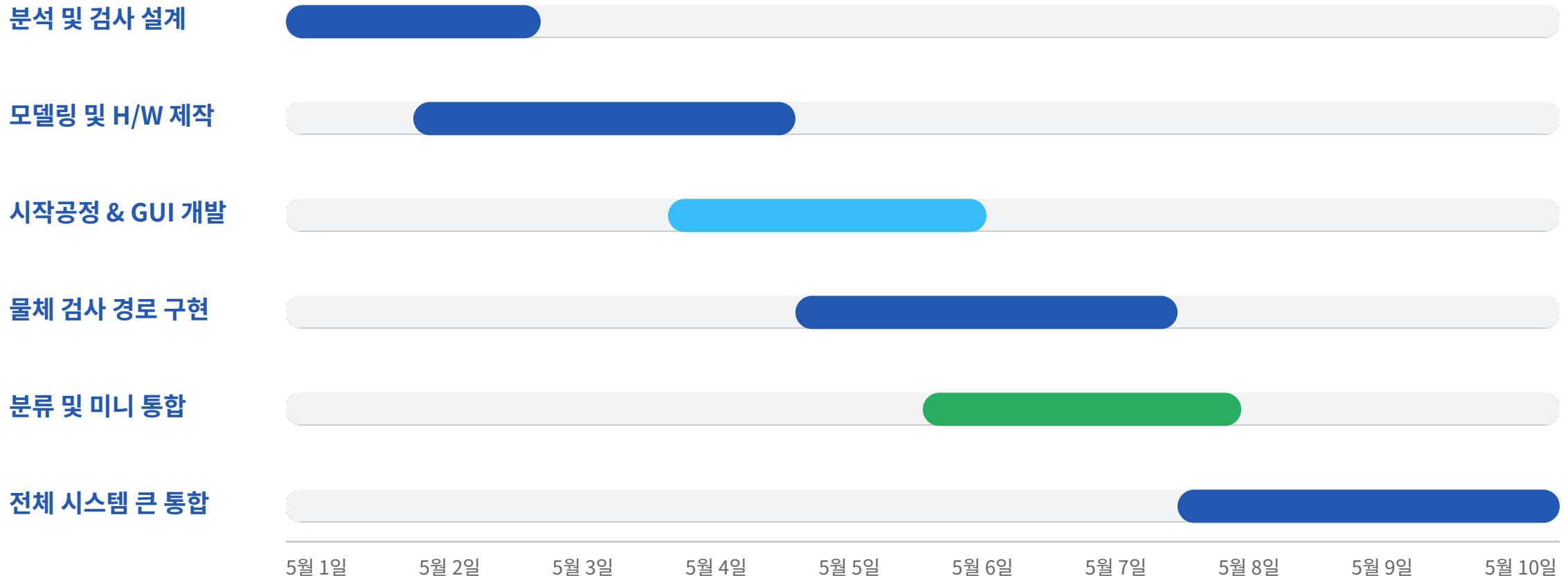
제현

윤제현

- 검사하기 로직 구축
- 데이터 수집 및 필터링
- 표면검사 알고리즘
- 분석 리포트 생성

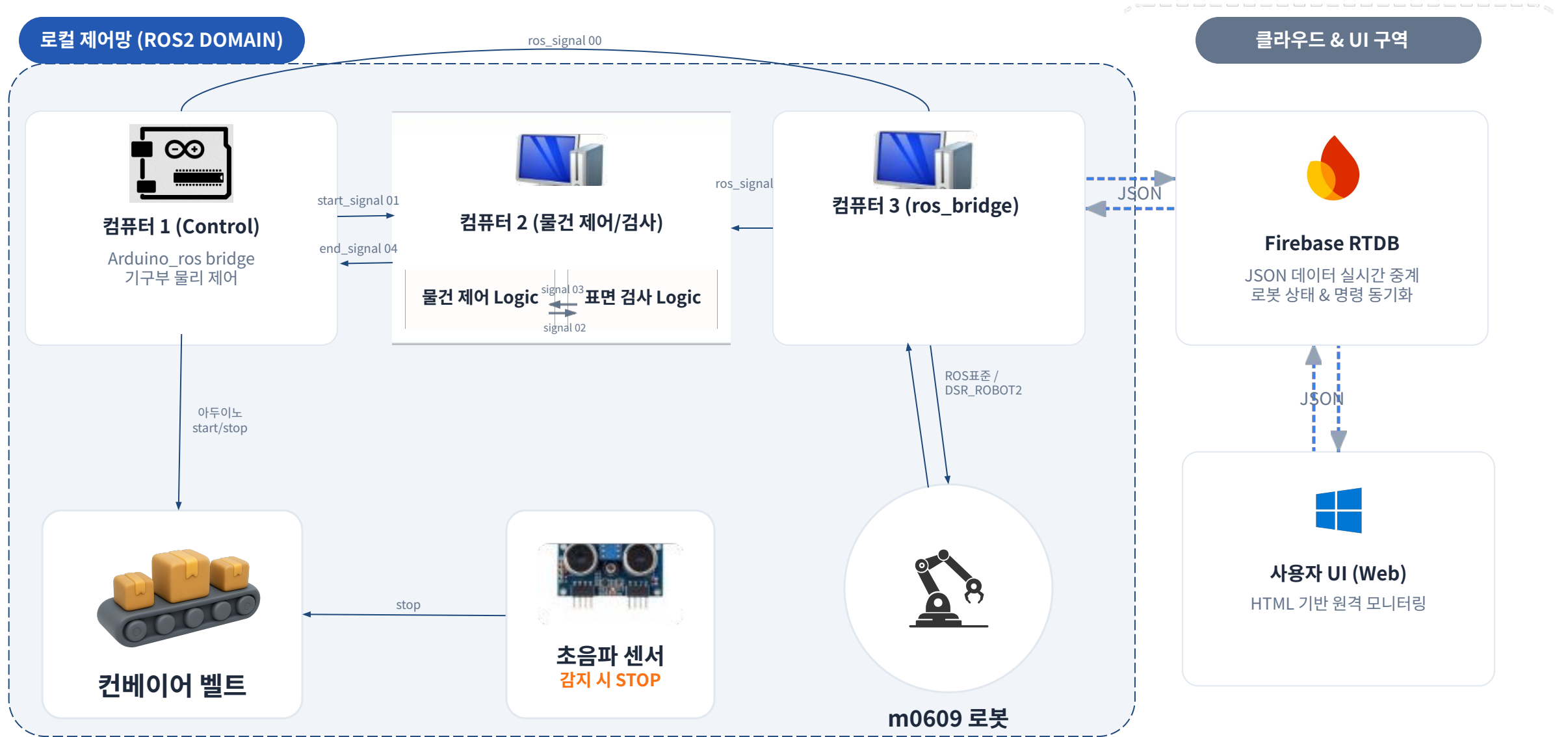
"각 분야의 전문성을 바탕으로 유기적인 협업을 통해 비전-프리(Vision-Free) 검수 시스템을 완성하였습니다."

## 03 프로젝트 수행 절차 및 방법 | 개발 일정

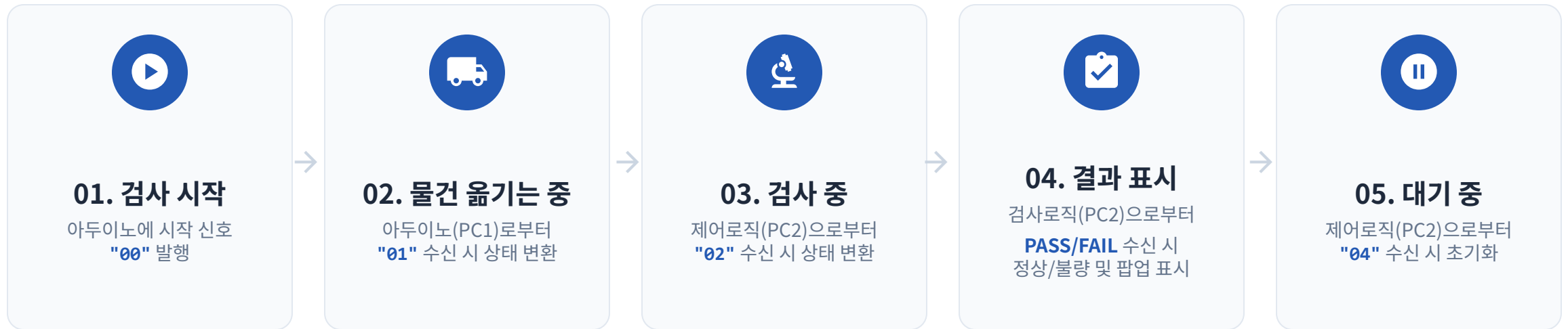


 **Key Milestone:** 5월 1일 기초 분석을 시작으로 5월 5일 미니 통합을 거쳐, 5월 10일 최종 '큰 통합' 공정 완료까지 총 10일간 유기적으로 진행되었습니다.

# 04 프로젝트 수행 경과 | 시스템 아키텍처 구성도



## | 04 프로젝트 수행 경과 | GUI 전체 모니터링 시나리오



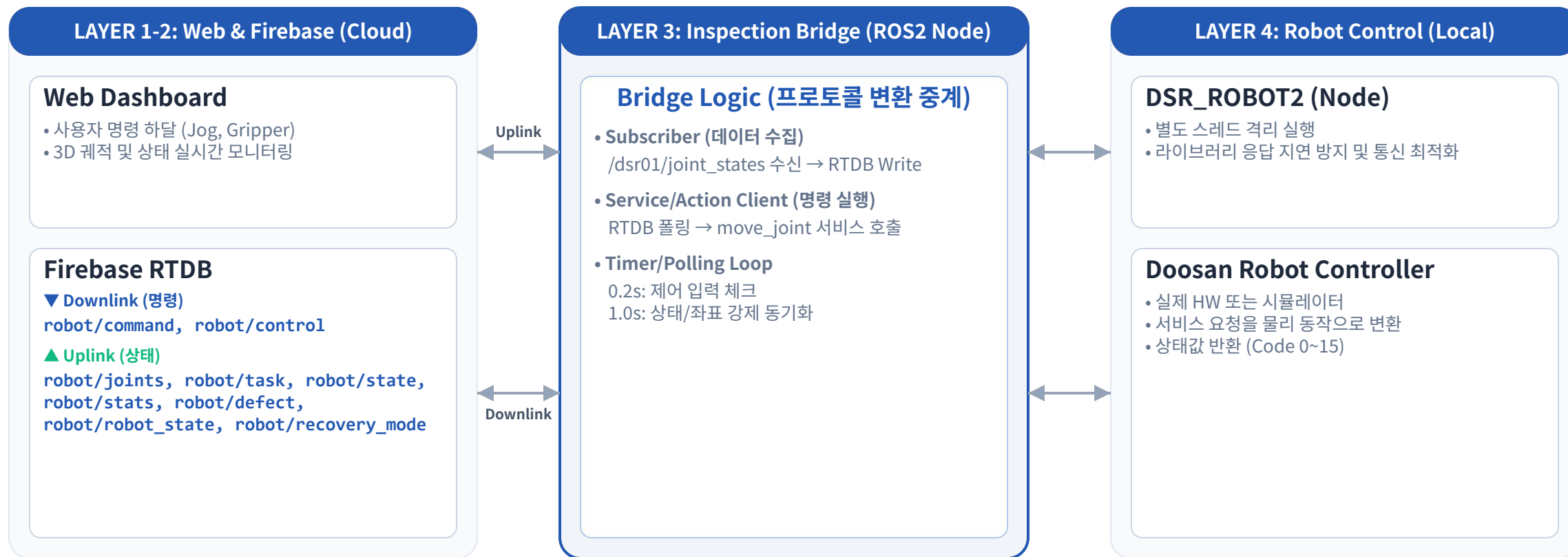
### [전체 시나리오 상세]

본 시나리오는 "00(시작) → 01(운송) → 02(검사) → PASS/FAIL(판정) → 04(종료 및 초기화)"로 이어지는 전형적인 스마트 팩토리 제어 루프를 따르고 있습니다.

사용자가 00(시작) 신호를 인가하면 컨베이어가 구동되며 전체 공정이 활성화됩니다. 아두이노 센서를 통해 물체가 감지되면 01(운송) 신호를 송신하고 이를 수신받은 제어로직이 물건 이송을 시작하며, 완료 후 02(검사) 명령을 하달합니다. 검사 로직에 따라 산출된 PASS/FAIL 판정 결과는 실시간 상태 전환 및 팝업 알림으로 출력되며, 최종적으로 제어로직이 물건을 분류이송까지 완료하면 04(완료) 신호를 통해 시스템이 다시 대기 상태로 복귀하는 유기적인 제어 구조를 가집니다.

# 04 프로젝트 수행 경과 | 시스템 노드 아키텍처 설계 개요

## 하이브리드 클라우드 제어 아키텍처 : 원격 제어 및 디지털 트윈 구현



### 주요 데이터 흐름 시나리오

- 모니터링(Uplink): 로봇(dsr01) → 브리지 노드(Callback) → Firebase(RTDB) → 웹 대시보드(3D 시각화)
- 제어(Downlink): 웹(명령) → Firebase(RTDB) → 브리지 노드(Polling) → 로봇 서비스(Action)
- 예외/복구: 로봇(보호정지) → 브리지 노드(감지) → 웹(복구 활성화) → 브리지 노드(Service Call) → H2R 실행

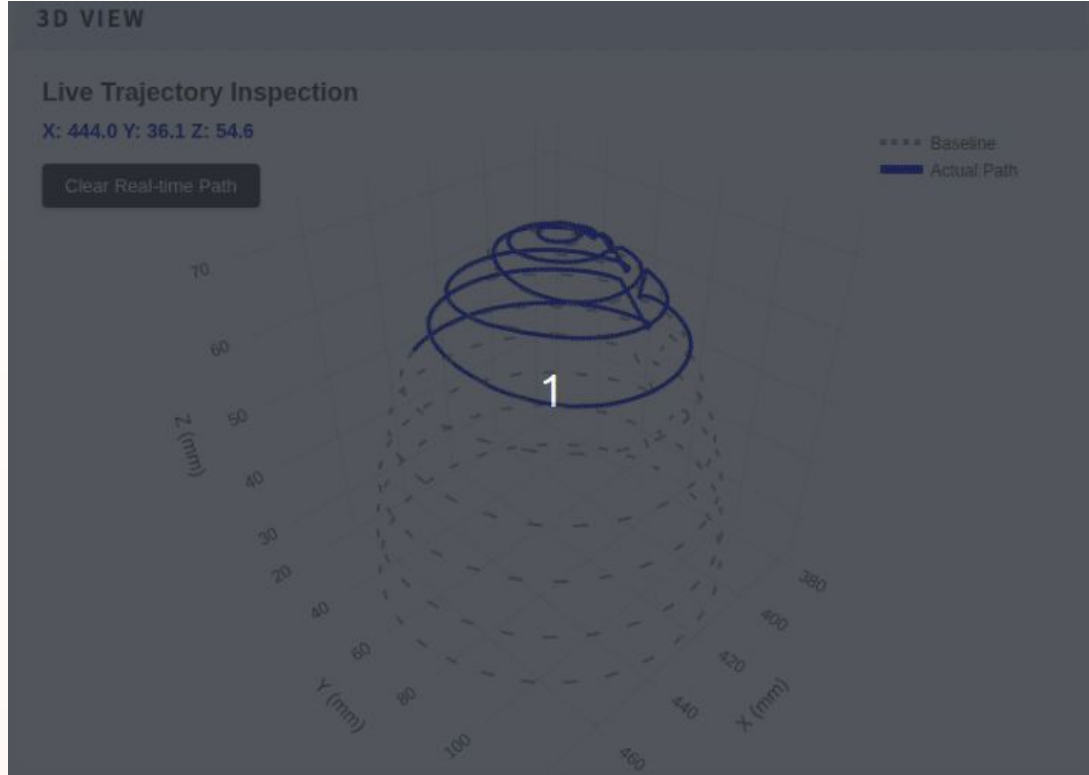
# 04 프로젝트 수행 경과 | 웹페이지 구성

웹 기반 로봇 제어 및 실시간 모니터링 대시보드 인터페이스 레이아웃

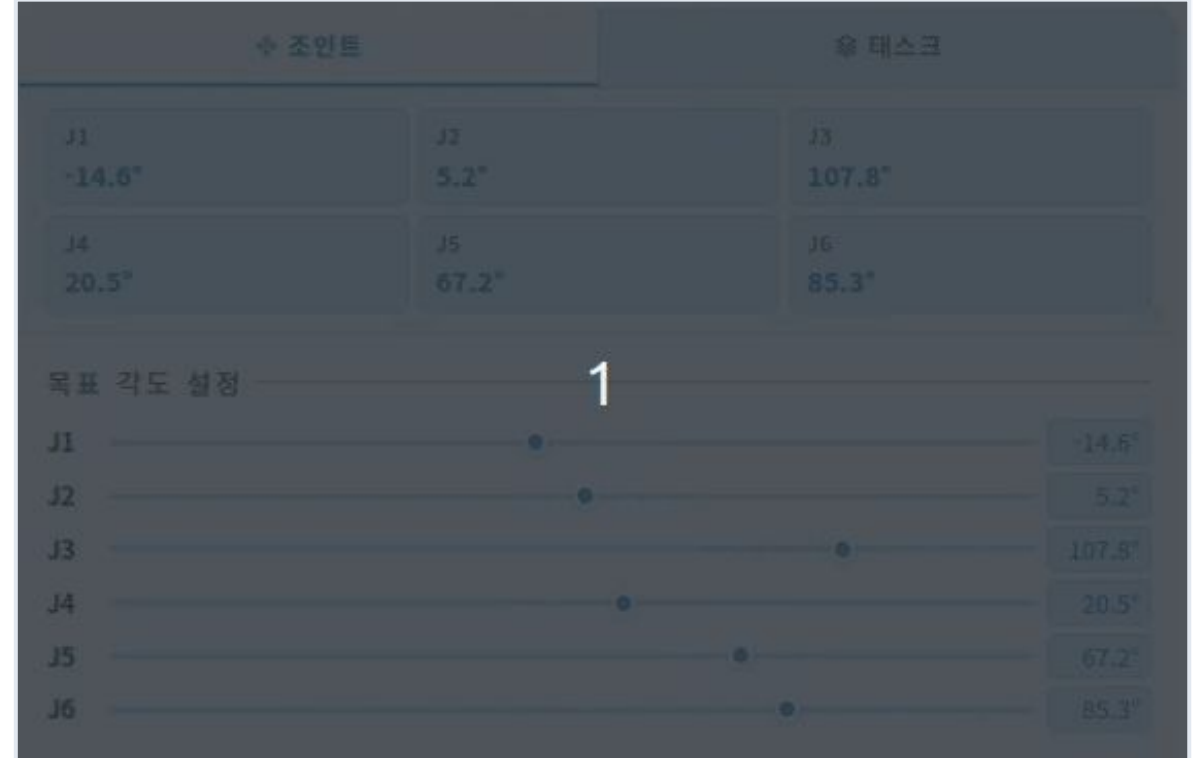


## 04 프로젝트 수행 경과 | Firebase RTDB를 사용한 실시간 모니터링

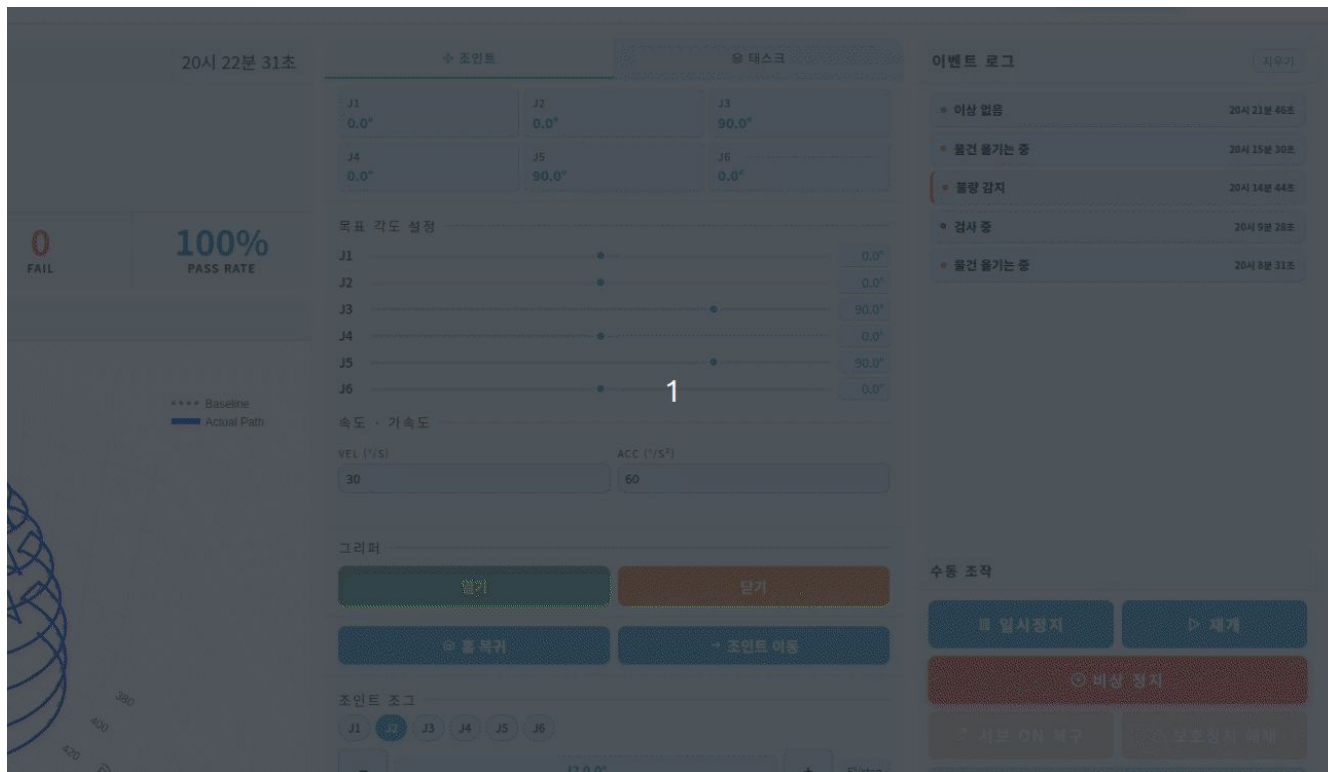
RTDB Path: robot/dsr01/current\_pose



RTDB Path: robot/joints & robot/task



## 04 프로젝트 수행 경과 | Firebase Storage를 사용한 Error Log 재확인



로그 클릭 가능

### 이력 확인 기능

#### • 실시간 로그 연동

이벤트 로그 영역에서 발생한 불량 기록을 실시간으로 확인하고 관리할 수 있습니다.

#### • 팝업 상세 조회

특정 로그를 클릭하면 별도의 팝업창을 통해 해당 시점의 상세 데이터와 이전 불량 기록 조회가 가능합니다.

#### • 데이터 시각화 및 조작

3D View와 차트를 통해 과거의 궤적 및 조인트 상태를 입체적으로 분석할 수 있으며, 마우스 컨트롤을 통해 다각도에서 확인이 가능합니다.

## | 04 프로젝트 수행 경과 | 멀티스레딩 아키텍처 설계 (5개 스레드)



### Thread 1: 메인 제어

- 검사 신호 및 결과(PASS/FAIL) 수신
- 로봇 조인트/태스크 상태 토픽 발행
- Firebase 제어 명령 폴링 (0.2초 주기)
- 조그(Jog) 및 그리퍼 제어 서비스 호출



### Thread 2: DSR Executor

- DSR\_ROBOT2 전용 노드 스핀
- 두산 로봇 API(get\_state 등) 응답 처리
- DRL 스크립트 정지 및 라이브러리 연동
- 메인 스레드와의 격리



### Thread 3-5: 복구 전담 interrupt

- 복구 시퀀스 담당하는 데몬스레드
- 개별 호출 시 생성, 메인 스핀의 지연 방지

## | 04 프로젝트 수행 경과 | 멀티스레딩 아키텍처의 핵심 설계



### 지연 없는 제어

복구 작업 중에도 메인 스핀이 멈추지 않아  
비상정지 신호를 실시간으로 감지할 수  
있습니다.



### 격리된 실행

DSR 전용 Executor를 통해 라이브러리  
내부 응답 지연이 전체 시스템의 통신  
주기에 영향을 주지 않도록 설계했습니다.



### 상태 동기화

웹 최초 기동 시 로봇의 현재 좌표와 상태를  
Firebase와 동기화하여 웹 UI와의 데이터  
불일치를 원천 차단합니다.

## | 04 프로젝트 수행 경과 | 다양한 예외처리 시퀀스 구현

### 비상정지 및 서보 복구

긴급 상황 발생 시 즉각적인 시스템 정지 및 안전이 확보된 후의 서보 ON 복구 프로세스



### 보호정지 감지 및 안전 복구

- 외부 충돌이나 외력에 의한 안전 잠금 감지 시, 안전 복구 모드(Recovery Mode) 진입
- MovelH2r/ MovejH2r 제어를 통한 외력 제거 후 정상 모드 복귀 및 잠금 해제



### 공정 일시정지 및 작업 재개

로봇 모션 및 검사 진행 상황을 유지한 상태에서의 일시정지(Pause) 및 재개(Resume)



**원클릭 웹 복구 프로세스:** 웹 UI에서 버튼 하나로 복구 모드 진입부터 서보 ON, 외력 제거까지 티치펜던트의 역할을 완벽히 수행합니다.

# 04 프로젝트 수행 경과 | 예외처리 시퀀스별 핵심 API 및 ROS2 통신 구조

## SCENARIO 01

### 비상정지 및 서보 복구

DSR\_ROBOT2

drl\_script\_stop

스크립트 즉시 강제 종료

SRV

/system/servo\_off

서보 전원 즉시 차단

SRV

/system/set\_robot\_control

Command 3: 서보 활성화

## SCENARIO 02

### 보호정지 및 외력 제거

SRV

/system/set\_safety\_mode

RECOVERY 모드 진입

ACTION

/motion/move[j/l]\_h2r

Hand-to-Robot 미세 제어

SRV

/system/set\_safety\_mode

AUTONOMOUS 모드 복귀

SRV

/system/set\_robot\_control

Command 2: 보호정지 해제

## SCENARIO 03

### 일시정지 및 작업 재개

SRV

/motion/move\_pause

현재 궤적 유지 및 홀딩

SRV

/motion/move\_resume

홀딩 시점부터 작업 재개

# 04 프로젝트 수행 경과 | 검사 중단 및 시스템 초기화를 위한 최적화 시퀀스 설계

## PROBLEM IDENTIFICATION

### 기존 제어 명령의 한계점 분석

#### Case A: move\_pause

기존 큐 점유로 인한 인터럽트 불가능, 강제 재개로 인한 초기화 불가

#### Case B: move\_stop

단일 모션만 정지, 후속 큐의 연쇄 실행으로 인한 '동작 흘러내림' 발생

## TECHNICAL BENEFITS

### 하이브리드 방식 채택의 기대 효과

제어 무결성

운영 유연성

안전성 확보

완벽한 공정 종료, OEE 향상 및 고스트 모션 방지를 통한 시스템 신뢰도 극대화

최적화된 '검사중지' 3단계 시퀀스 (Proposed Solution): 공정의 전면 취소와 안전한 초기화를 위해 [Pause → Kill → Reset] 메커니즘을 적용했습니다.



### Step 1. Pause

#### move\_pause

신호 수신 즉시 로봇의 물리적 움직임을 멈춰 즉각적인 안전을 확보합니다.



### Step 2. Kill

#### drl\_script\_stop

대기 중인 모든 모션 큐를 강제 삭제(Flush)하여 잔류 명령 간섭을 차단합니다.



### Step 3. Reset

#### Home Return

작업 맥락이 제거된 상태에서 독립적으로 복귀하여 차기 사이클 대기 상태로 전환합니다.

## 04 프로젝트 수행 경과 | 모델링

---

### 그리퍼 툴

표면 스캐닝의 핵심 파트

### 그리퍼 툴 꽃이

툴 교체용 꽃이

### 사출물 툴

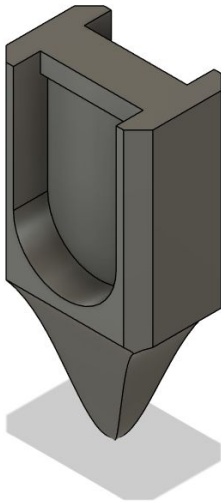
검사 대상 고정 지그

### 바깥 툴

전체 시스템 구조물

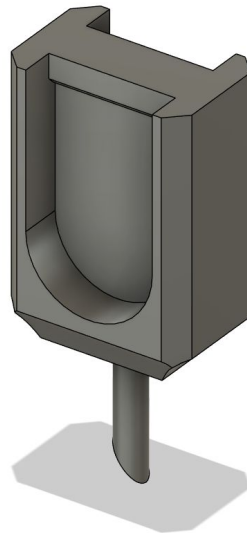
## 04 프로젝트 수행 경과 | 그리퍼 툴 (Gripper Tool)

V1. 초기 모델



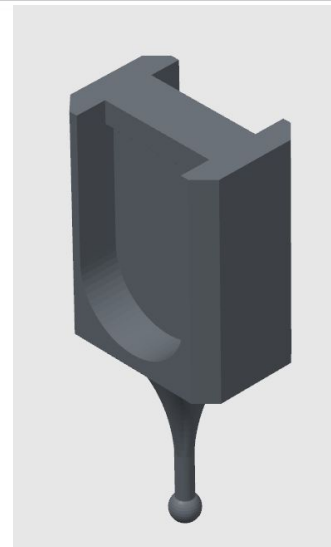
초기 디자인 그리퍼

V2. 개선 모델



팁 형태 보완

V3. 개선 모델



삼각뿔형 강성 강화

접촉 면적과 검사물 표면에 생성된 결함을 탐지하기 적합한 형태를 찾기 위해 툴의 여러 최적화를 진행한 결과 초기 모델로 진행

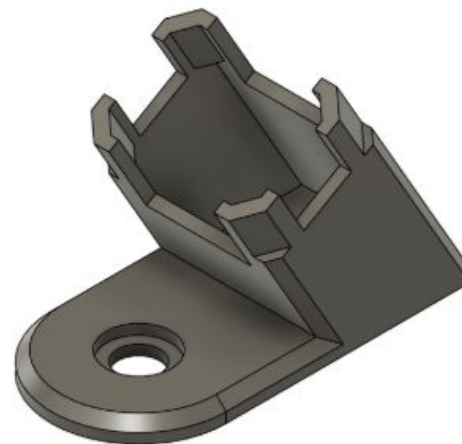
## 04 프로젝트 수행 경과 | 그립퍼 꽃이 (Gripper Holder)

Basic Design



구조 확인용 프로토타입

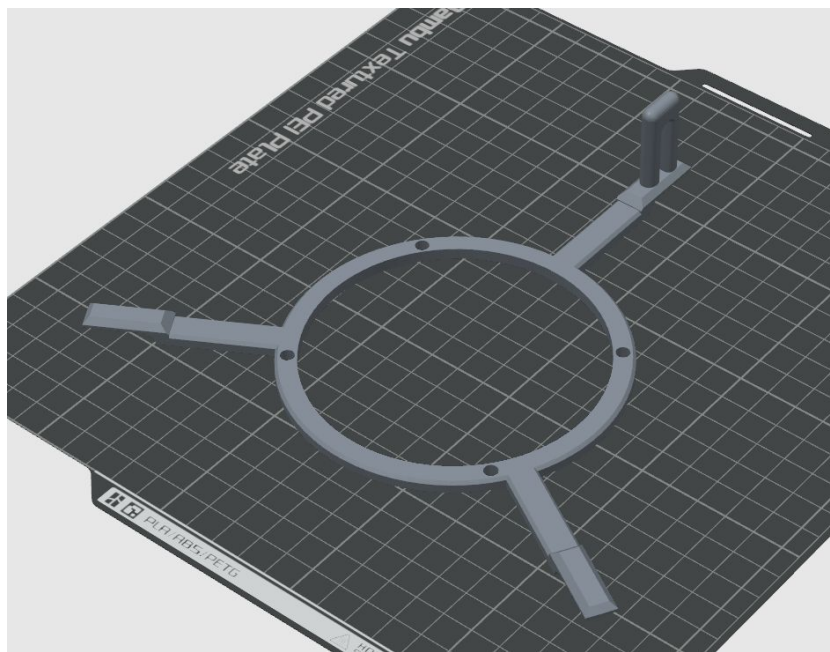
Final Design



체결력 및 렌더링 최적화

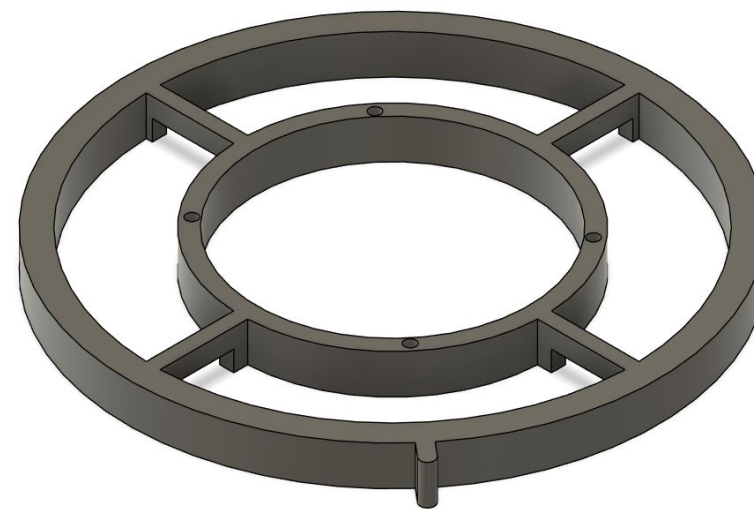
## 04 프로젝트 수행 경과 | 사출물 틀 (Injection Mold Frame)

Arc Component A



초기 사출물 틀 디자인

Arc Component B



최종 사출물 틀 디자인

## 04 프로젝트 수행 경과 | 바깥 틀 (Outer Frame)

Main Structure



3지 지지대 구조

Base Plate Integration

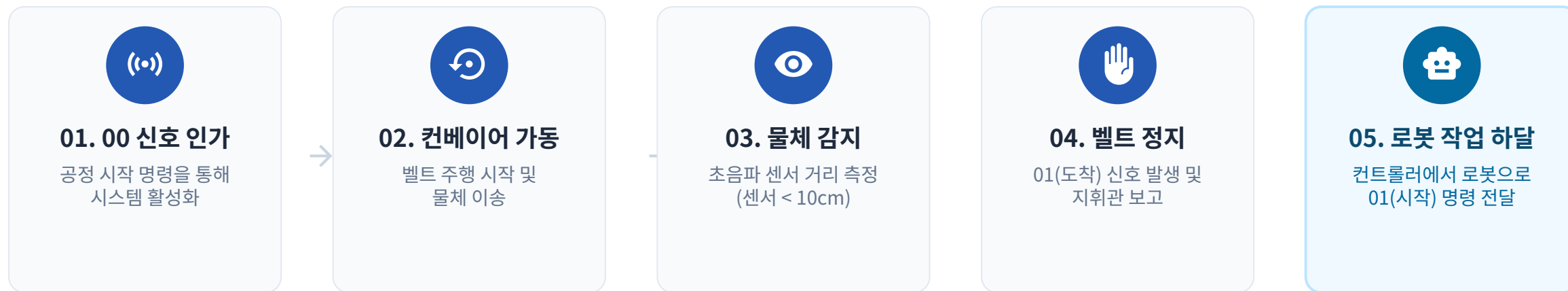


슬라이서(Bambu PEI Plate) 배치도

# 04 프로젝트 수행 경과 | Design Summary

| 카테고리   | 주요 개선 사항         | 결과             |
|--------|------------------|----------------|
| 그리퍼 툴  | 구형 -> 삼각뿔형 팁 변경  | 스캐닝 정밀도 향상     |
| 그리퍼 꽃이 | 체결 부위 보강 및 공차 조정 | 로봇 암 결합 안정성 확보 |
| 사출물 틀  | 곡률 정밀화 및 분할 설계   | 다양한 크기 대응 가능   |
| 바깥 틀   | 지지대 배치 최적화       | 시스템 전체 강성 증대   |

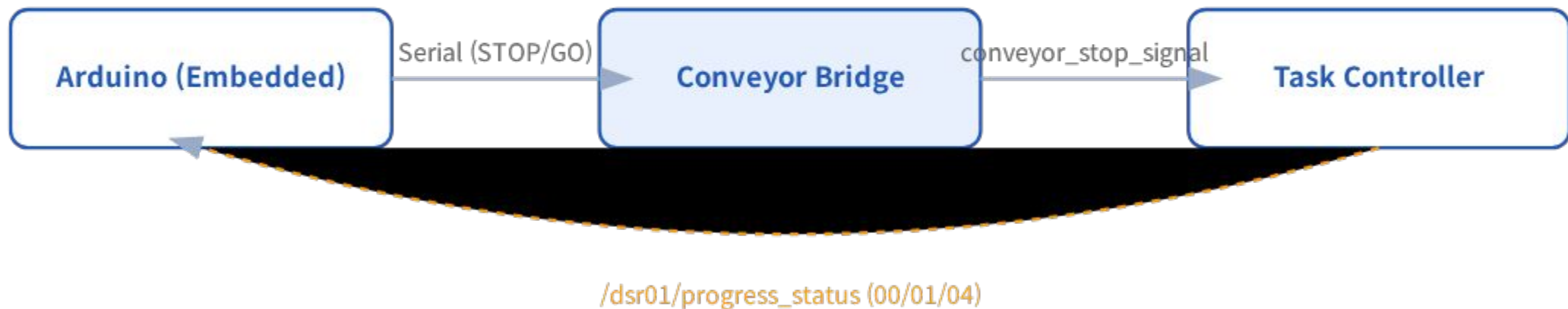
## 04 프로젝트 수행 경과 | 전체 시스템 제어 시나리오



### [전체 시나리오 상세]

사용자가 **00 신호**를 인가하면 컨베이어가 회전하며 공정이 시작됩니다. 초음파 센서에 물체가 감지되면 벨트가 즉시 멈추고 **01(도착) 신호**를 보냅니다. 지휘관은 로봇의 상태를 체크하여 작업 가능 시 **01(시작)**을 명령하고, 작업 완료 신호 **04**가 들어오면 다음 사이클을 대기하는 유기적 구조입니다.

## 04 프로젝트 수행 경과 | ROS2 노드 통신 아키텍처



### [Topic] progress\_status

전체 공정의 상태(00, 01, 04)를 공유하는 메인 버스입니다. 모든 노드가 이 토픽을 구독하여 동기화됩니다.

### [Bridge] Serial Gateway

아두이노의 로우 데이터를 ROS2 메시지 형식으로 변환하거나, 상위 명령을 시리얼로 전달합니다.

## 04 프로젝트 수행 경과 | 임베디드 로직: Arduino & Sensor

### 🔗 모터 제어 알고리즘

- 플래그로 초기 대기 상태 유지 `systemStarted`
- 기반 Non-blocking 스텝 펄스 생성 `micros()`
- 시리얼 입력 수신 시에만 공정 활성화 `"00"`
- 물체 감지 시 즉각 멈춤 후 `Serial.println("STOP")`

### 📏 센서 및 인터럽트

- 를 통한 에코 신호 정밀 측정 `attachInterrupt`
- 200ms 주기의 트리거링으로 안정적인 거리 데이터 확보
- 히스테리시스 적용: 10cm 이하 정지 / 20cm 이상 재가동
- 상태 변수 로 중복 신호 방지 `objectDetected`

## | 04 프로젝트 수행 경과 | 브릿지 로직: Conveyor Sensor Bridge

**역할:** 아두이노(Hardware)와 Task Controller(Software) 사이의 실시간 통신 중계

### ↳ Downstream (ROS2 -> ARDUINO)

- `progress_status` 포착
- 아두이노 시리얼 포트에 바이트 데이터 전송 `b"00\n"`
- 하드웨어 레벨의 시스템 가동 트리거링

### ↳ Upstream (ARDUINO -> ROS2)

- 별도 스레드에서 시리얼 상 `readline()`
- 아두이노가 "STOP" 전송 시 (들 `"01"` 작) 발행
- 로봇 제어권을 가진 지휘관 노드에 이벤트 알림

## 04 프로젝트 수행 경과 | 지휘 로직: Task Controller

### 🔹 상태 기반 의사결정

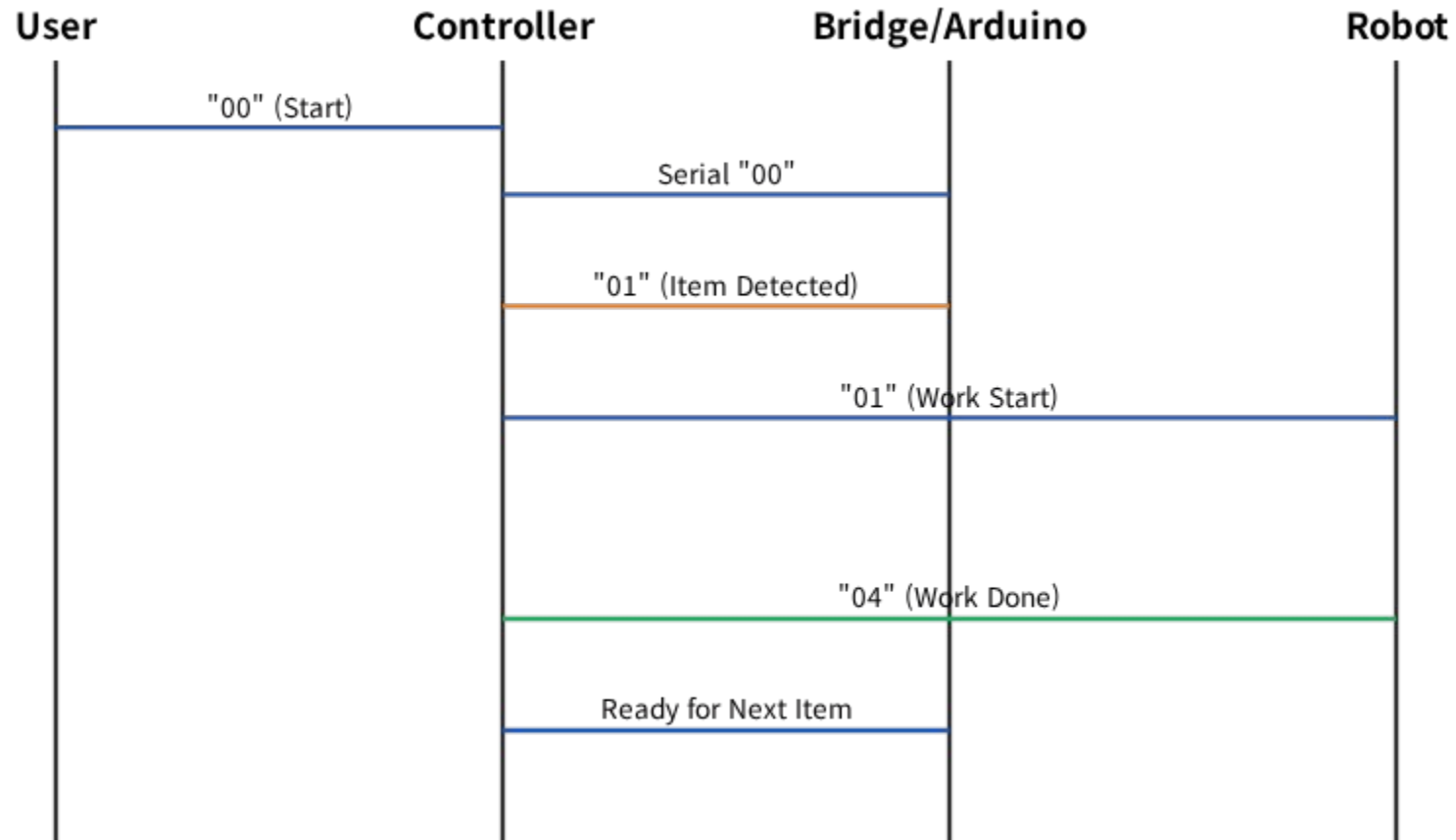
두 가지 플래그를 조합하여 최적의 작업 타이밍을 결정합니다.

- : 컨베이어로부터 물건 도착 보고 유무 `item_ready`
- : 로봇이 현재 작업 중인지 여부 `robot_busy`
- 조건: `item_ready AND NOT robot_busy`

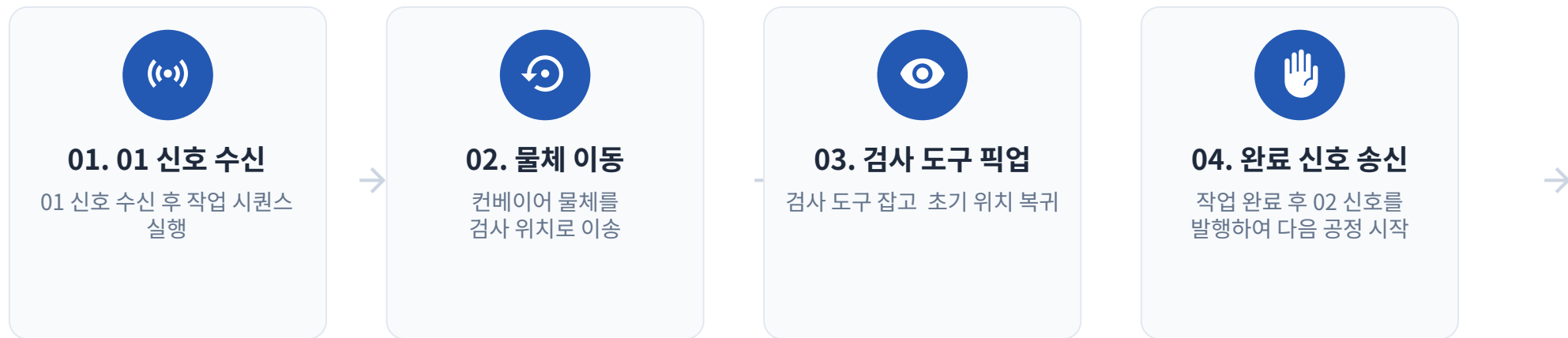
### 🔄 공정 사이클 관리

- 00 수신: 시스템 초기화 및 시작 준비
- 01 수신 (from Bridge): `item_ready = True`
- 작업 하달: 로봇에게 신호 발행 `"01"`
- 04 수신 (from Robot): `robot_busy = False`

## 04 프로젝트 수행 경과 | 시스템 제어 시퀀스 다이어그램



## 04 프로젝트 수행 경과 | 컨베이어벨트



### [전체 시나리오 상세]

컨베이어 벨트로부터 **01 신호**를 수신하면 컨베이어 벨트 위로 이동하면서 작업 시퀀스를 시작합니다. 그리퍼를 열고 물체를 잡아 검사 위치로 이동하여 물체를 내려놓고 도구 위치로 가 검사를 위한 도구를 잡아 초기 좌표로 복귀합니다. 모든 동작이 정상적으로 완료되면 **02 (공정 시작) 신호**를 보내어 다음 검사 공정이 시작될 수 있도록 합니다. 또한 작업 중에는 중복으로 들어오는 01 신호를 무시하도록 설정하여 동일 공정이 반복 실행되지 않도록 제어하였다.

## | 04 프로젝트 수행 경과 | 로봇 제어 시나리오



### 협동로봇 작업 시퀀스

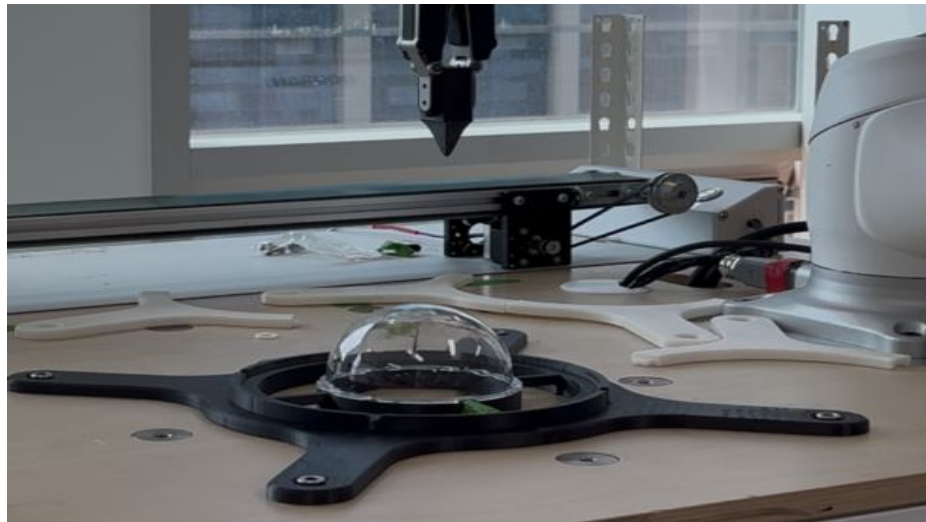
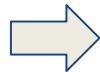
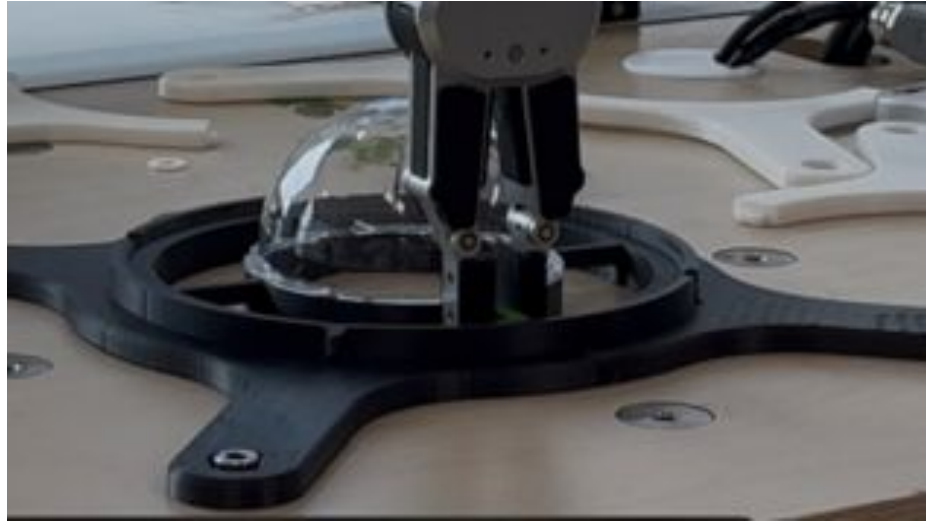
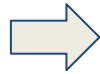
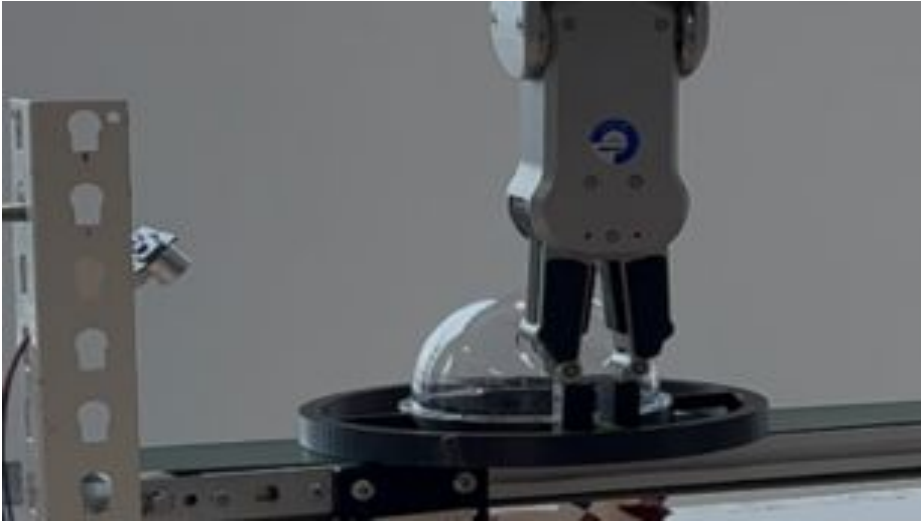
- **초기화:** 그리퍼 오픈 및 홈(Home) 위치 복귀
- **픽업:** 컨베이어 위 물체 감지 시 하강 및 그리핑
- **이송:** 검사 장소(Inspection)로 정밀 이동
- **검사 및 도구:** 물체 적재 후 작업용 도구 픽업
- **완료:** 작업 종료 신호(02) 송신 및 대기



**비동기 신호 처리 (Flag-based Logic):** `rclpy.spin_once`를 통한 비차단 방식 설계. `trigger_start` 플래그를 사용하여 ROS2 통신과 로봇 모션 제어 간의 병목 현상 방지.

**통신 프로토콜:** `START_SIGNAL("01")` 수신 시 작업 개시, `DONE_SIGNAL("02")` 송신으로 상위 지휘관 노드에 상태 보고.

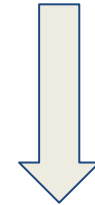
## | 04 프로젝트 수행 경과 | 공정 전 초기 정렬 흐름



## | 04 프로젝트 수행 경과 | 로봇 제어 노드 통신 아키텍처



START("01") Sub



검사 전 정렬 동작 실행

DONE("02") Pub

### [Node] M0609 Controller

핵심 로직: **spin\_once** 기반의 깃발(Flag) 제어 시스템

- **START\_SIGNAL("01")** 수신 시 작업 시퀀스 예약
- **run\_sequence()**: 홈→픽업→검사→도구 잡기 순차 실행

### [Topic] progress\_status

전체 시스템 동기화를 위한 양방향 메시지 버스

- **Sub**: 상위 시스템으로부터의 시작 명령("01") 상시 감시
- **Pub**: 모든 시퀀스 종료 후 완료 신호("02") 외부 송신

## 04 프로젝트 수행 경과 | 초기 정렬 순서 및 동작 방식

| 순서 | 동작 방식   | 작업 설명            | 사용하는 좌표/명령                     | 의미                        |
|----|---------|------------------|--------------------------------|---------------------------|
| 1  | movej   | 컨베이어 위 좌표 이동     | self.conveyor_above_pick_joint | 컨베이어 위 접근 위치로 관절 이동       |
| 2  | gripper | 그리퍼 열기           | "open"                         | 물체를 잡기 전에 그리퍼 열기          |
| 3  | movel   | 컨베이어 좌표 이동       | self.conveyor_pick_posx        | 컨베이어 위 물체 위치로 직선 하강       |
| 4  | gripper | 그리퍼 닫기           | "close"                        | 컨베이어 위 물체 잡기              |
| 5  | movel   | 컨베이어 위 좌표 복귀     | self.conveyor_above_pick_posx  | 물체를 잡은 뒤 컨베이어 위로 직선 상승    |
| 6  | movej   | 검사 장소 위 이동       | self.inspection_above_joint    | 검사 장소 위 접근 위치로 관절 이동      |
| 7  | movel   | 검사 장소 하강         | self.inspection_place_posx     | 검사 장소에 물체를 내려놓기 위해 직선 하강  |
| 8  | gripper | 그리퍼 열기           | "open"                         | 검사 장소에 물체 내려놓기            |
| 9  | movel   | 검사 장소 위 이동       | self.inspection_above_posx     | 물체를 내려놓은 뒤 검사 장소 위로 직선 상승 |
| 10 | movel   | 압입 지점 50mm 상부 이동 | self.pressing_above_posx       | 압입 위치 위쪽으로 직선 이동          |
| 11 | movel   | 압입 진행            | self.pressing_posx             | 압입 위치로 직선 하강              |
| 12 | movel   | 압입 지점 50mm 상부 이동 | self.pressing_above_posx       | 압입 후 다시 위로 직선 상승          |
| 13 | movej   | 그리퍼 위 장소 이동      | self.tool_safe_joint           | 그리퍼 장착/반납 위치 위로 관절 이동     |
| 14 | movel   | 그리퍼 좌표 이동        | self.tool_place_posx           | 그리퍼 실제 위치로 직선 접근          |
| 15 | gripper | 그리퍼 닫기           | "close"                        | 그리퍼를 잡거나 고정하는 동작          |
| 16 | movel   | 그리퍼 위 장소 복귀      | self.tool_safe_posx            | 그리퍼 위치에서 위로 직선 탈출         |
| 17 | movej   | 초기 위치 이동         | self.home_joint                | 홈 위치로 관절 이동               |

## 04 프로젝트 수행 경과 | 이런 좌표를 설정한 이유

핵심 이유

로봇 이동 시 주변 시설물 및 환경과의 물리적 충돌 방지

Line 5

컨베이어 위 좌표 복귀

```
self.conveyor_above_pick_posx
```

물체를 잡은 뒤 컨베이어 위로 직선 상승하여 벨트 프레임이나 주변 구조물에 걸리지 않도록 안전 고도 확보

Line 9

검사 장소 위 이동

```
self.inspection_above_posx
```

물체를 내려놓기 전 검사 장소 위로 수직 이동하여 물체를 고정하는 틀과의 물리적 충돌 방지

Line 13

그리퍼 위 장소 이동

```
self.tool_safe_joint
```

그리퍼 장착 및 반납을 위해 관절 각도를 조절하여 로봇 암이 작업대 내부 설비와 부딪히지 않는 경로로 이동

## 04 프로젝트 수행 경과 | 좌표 설정 안 했을 때



## 04 프로젝트 수행 경과 | 좌표 설정 했을 때

---



## | 04 프로젝트 수행 경과 | MOVEJ VS MOVEL

| 구분       | 사용된 Move | 주요 목적 및 동작                          |
|----------|----------|-------------------------------------|
| 큰 자세 이동  | movej    | 초기 좌표 복귀, 도구 반납 접근, PASS/FAIL 분배 이동 |
| 직선 접근/스캔 | movel    | 검사 장소 위 상승, 직선 Scratching 스캔, 도구 안착 |
| 그리퍼 제어   | gripper  | 도구 내려놓기/잡기, 컨베이어 물체 파지              |

## 04 프로젝트 수행 경과 | trigger\_start 비동기 실행 구조

### 비동기 신호 처리 변수: trigger\_start

#### 1. Callback과 실행부의 역할 분리

메시지 수신 시 즉시 로직을 실행하지 않고, 플래그만 설정하여 시스템 부하를 방지합니다.

- **Topic Callback**: 시작 신호(01) 수신 여부만 체크 후 기록
- **Main Loop**: 기록된 플래그를 확인하여 실제 로봇 시퀀스 수행

#### 2. 효율적인 시퀀스 처리 구조

비동기적으로 들어오는 데이터 신호를 동기적인 메인 루프 시퀀스로 변환합니다.

- 메인 루프에서 **trigger\_start**가 True일 경우에만 시퀀스 로직 진입
- 작업 시작 시 플래그 초기화 및 로봇 제어 권한 획득

#### Logic Flow

```
# Topic Callback
if (msg == '01'):
    trigger_start = True

# In Main Loop
while True:
    if trigger_start:
        trigger_start = False
        execute_robot_work()
        spin_once()
```

\* 이 구조는 콜백 함수가 로봇 동작 중 지연되는 것을 방지하여, 실시간 메시지 수신 성능을 보장합니다.

## 04 프로젝트 수행 경과 | 실행 상태 관리

### 상태 제어 변수: `is_running`

#### 1. 중복 신호 필터링 및 트리거 제어

네트워크 지연이나 센서 특성으로 인해 동일한 시작 신호가 반복 유입될 수 있습니다.

- 신호 수신 시 `is_running`이 `False`일 때만 시퀀스 시작 트리거 활성화

#### 2. 시퀀스 생명 주기 관리 (Lifecycle)

Main Loop에서 시퀀스 동작의 시작과 끝을 명확히 정의합니다.

- 시퀀스 진입 즉시 `is_running = True` 설정
- 모든 동작 완료 후 `is_running = False` 복귀

#### Logic Flow

```
if (msg == '01'):  
    if not is_running:  
        trigger_start = True
```

```
# In Main Loop  
is_running = True  
execute_sequence()  
is_running = False
```

\* 이 로직을 통해 로봇이 동작 중일 때 발생하는 추가적인 명령 입력을 원천적으로 무시하여 시스템의 안전성을 확보합니다.

## 04 프로젝트 수행 경과 | 실행 상태 관리

### 이를 통해 얻을 수 있는 기대 효과

#### 1. 중복 실행 방지

로봇이 검사 위치로 이동 중이거나 검사 중일 때, 새로운 시작 신호(01)가 수신되더라도 현재 시퀀스를 보호하여 중복 명령으로 인한 오작동을 원천 차단합니다.

#### 2. 로봇 동작의 안전성 확보

물체 파지 후 이동 단계에서 컨베이어 벨트로 다시 돌아가는 역동작을 방지하여, 충돌 위험을 제거하고 전체 공정의 물리적 안정성을 극대화합니다.

#### 핵심 제어 로직 요약

**IF** (데이터 == '01') **AND** (is\_running == **False**) **THEN** trigger\_start = **True**

\* 로봇이 이미 실행 중(is\_running = True)일 경우, 수신된 모든 시작 신호는 시스템에서 무시됩니다.

## 04 프로젝트 수행 경과 | 컨베이어 벨트 위 그리퍼 좌표 설정



### 어떤 문제점?

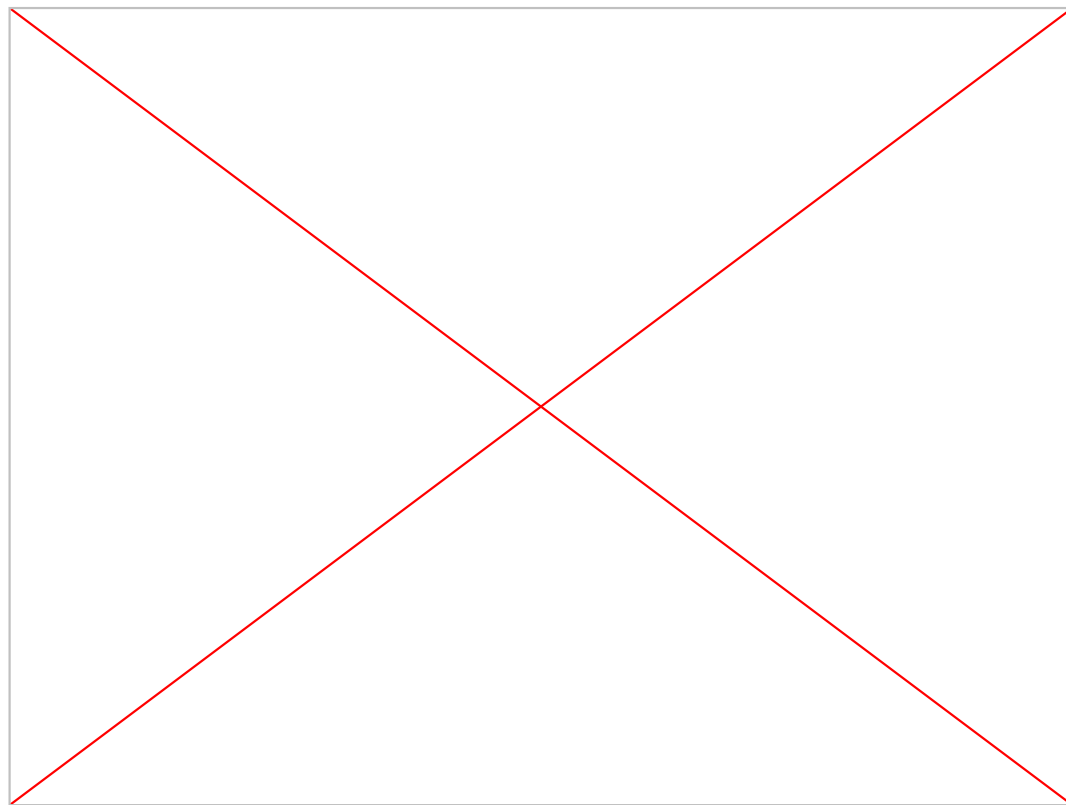
초음파 센서의 특성상 컨베이어 벨트 위의 물체가 **항상 일정한 위치에서 정지하지 않음**으로 인해 발생하는 정밀도 저하 문제

### 어떻게 해결?

반복 실험을 통해 객체 정지 범위의 **최대·최소 좌표를 산출**하고, 해당 범위를 모두 포괄하는 **최적 그리퍼 좌표를 설정**하여 파지 성공률 극대화

## 04 프로젝트 수행 경과 | 컨베이어 벨트 위 그리퍼 좌표 설정

---



9번의 실험 중 9번 모두 정해둔 그리퍼의 x축 사이로 오는 것 확인!!

## | 04 프로젝트 수행 경과 | 검사 위치에서의 pressing

왜 pressing이 필요한가?

### 원인 01

컨베이어 벨트 위 물체 정지 위치의 **오차 발생**

### 원인 02

그리퍼가 물체를 잡는 위치에도 **오차 발생**



### 해결 방안: Pressing

항상 검사 위치에 완벽한 배치가 불가능한  
한계를 극복

**Pressing을 통해 물체가 항상 검사 틀 안으로  
들어가도록 구현**

## 04 프로젝트 수행 경과 | 검사 위치에서의 pressing

---

왜 pressing이 필요한가?

컨베이어 벨트에서 물체 멈추는 위치의 오차 발생



그리퍼가 잡는 위치에도 오차 발생



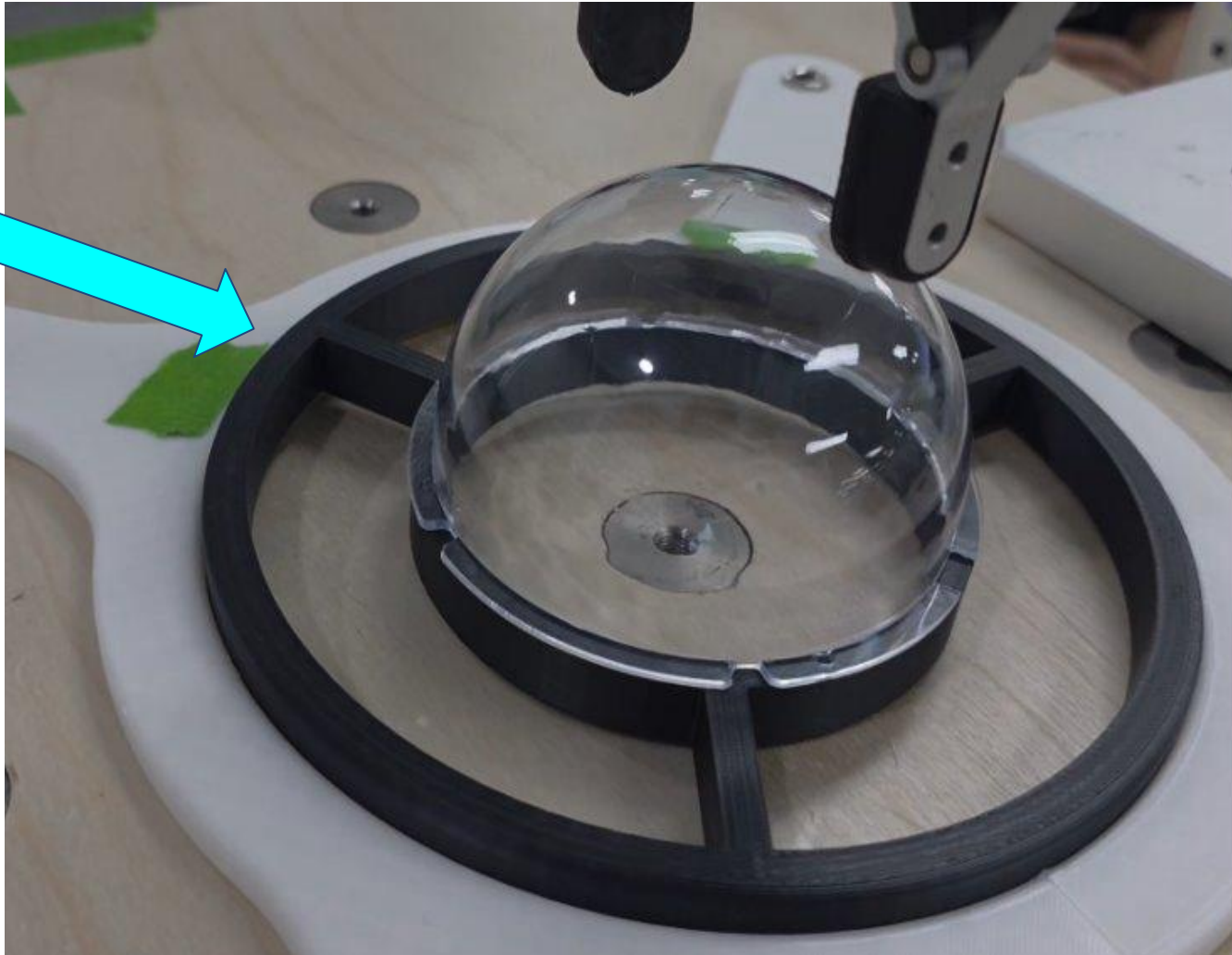
항상 검사 위치에 완벽한 배치 불가능



Pressing을 통해 항상 검사 틀 안으로 들어가도록 구현

## 04 프로젝트 수행 경과 | 검사 위치에서의 pressing

pressing의 필요성

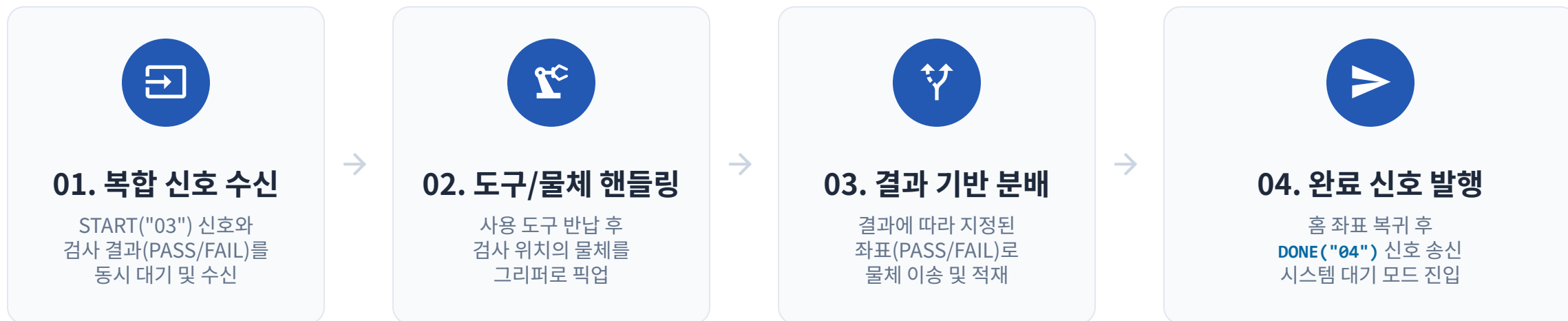


## 04 프로젝트 수행 경과 | pressing 구현 영상

---



## | 04 프로젝트 수행 경과 | PASS/FAIL 분배 시스템 제어 시나리오



### [last\_work\_final.py 핵심 로직 상세]

- **신호 동기화 제어:** `spin_once` 기반의 비차단(Non-blocking) 루프를 통해 시작 신호("03")와 비전 검사 결과(PASS/FAIL)가 모두 수신될 때까지 대기하며, 중복 실행 방지 플래그(`is_running`)를 활용합니다.
- **스마트 모션 인터럽트:** `wait_until_stopped()` 함수를 통해 로봇의 Idle 상태를 0.1초 단위로 감시하여 동작 완료 여부를 정밀하게 판정합니다.
- **리스트 기반 태스크 관리:** 도구 반납부터 물체 픽업, 분배 적재까지의 전 과정을 리스트 형태의 `tasks`로 정의하여 공정의 확장성과 가독성을 확보하였습니다.

## 04 프로젝트 수행 경과 | 공정 분류 시퀀스 상세

### 분류 순서 및 동작 메커니즘

#### Step 01. 초기화 및 도구 반납

홈 좌표 복귀 후 사용한 도구를 지정된 위치에 반납합니다.

`tool_place_posx | open`

#### Step 02. 검사 대상 물체 픽업

그리퍼를 열고 검사 위치로 하강하여 물체를 파지합니다.

`inspection_place_posx | close`

#### Step 03. 결과 기반 분배 이송

수신된 결과에 따라 PASS 또는 FAIL 적재함으로 이송합니다.

`pass_pos_joint / fail_pos_joint`

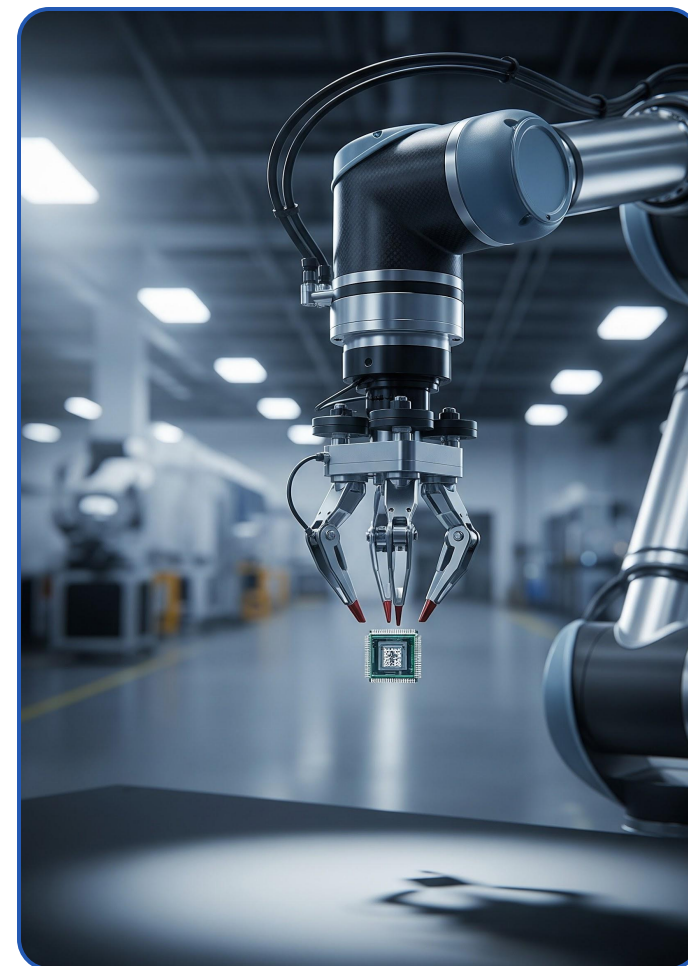
#### Step 04. 시퀀스 완료 및 신호

물체 적재 후 홈 좌표로 복귀하며 완료 신호 ("04")를 발행합니다.

`DONE_SIGNAL ("04")`

#### 💡 스마트 모션 제어 로직 (wait\_until\_stopped)

- movej/movel 호출 후 로봇의 Idle 상태(State 1)를 0.1초 단위로 실시간 감시
- 0.5초 이상 정지 상태 유지 시에만 다음 태스크로 전환하여 물리적 안정성 확보



# 04 프로젝트 수행 경과 | 공정 후 시스템에서의 노드 통신 흐름

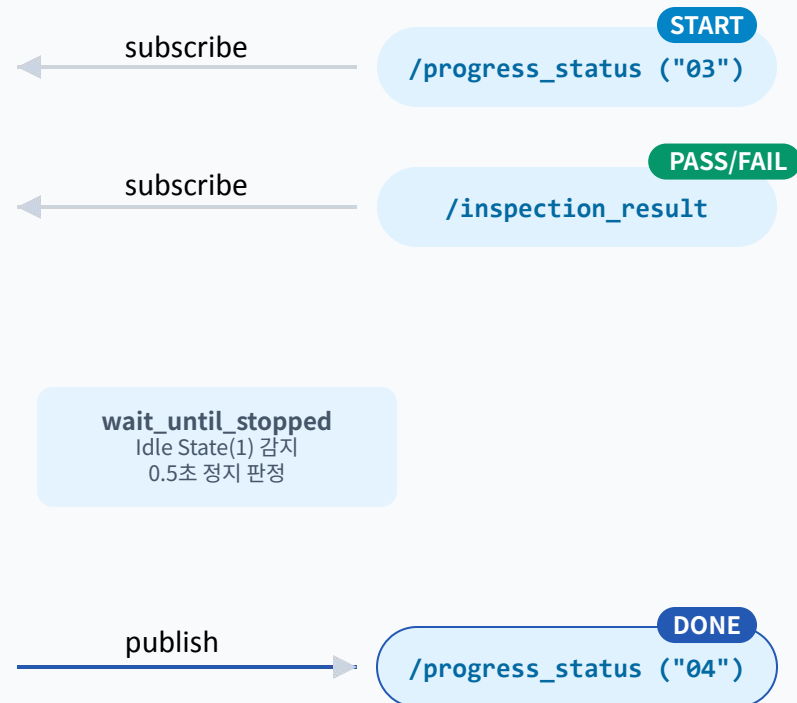
## 노드 통신 흐름

### M0609SortingTestNode

- **is\_running**: 시퀀스 중복 방지
- **pending\_result**: 결과 보관

#### 핵심 시퀀스 관리

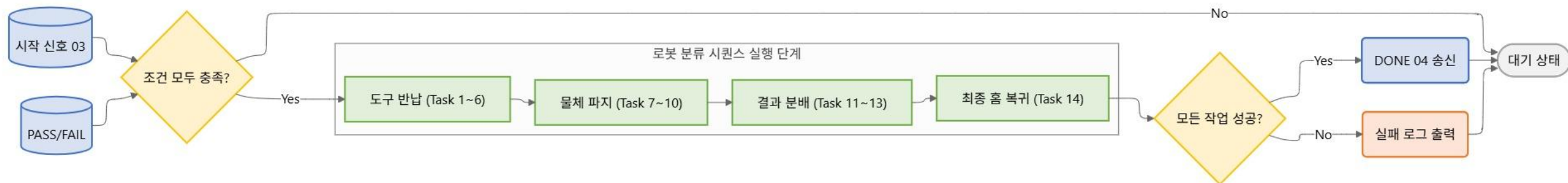
- **spin\_once()**: 상태 감시
- **tasks**: 리스트 기반 태스크
- **wait\_until\_stopped**: 0.5s
- **DONE**: 완료 신호 발행



## 04 프로젝트 수행 경과 | 공정 후 분류 순서 및 동작 방식 상세

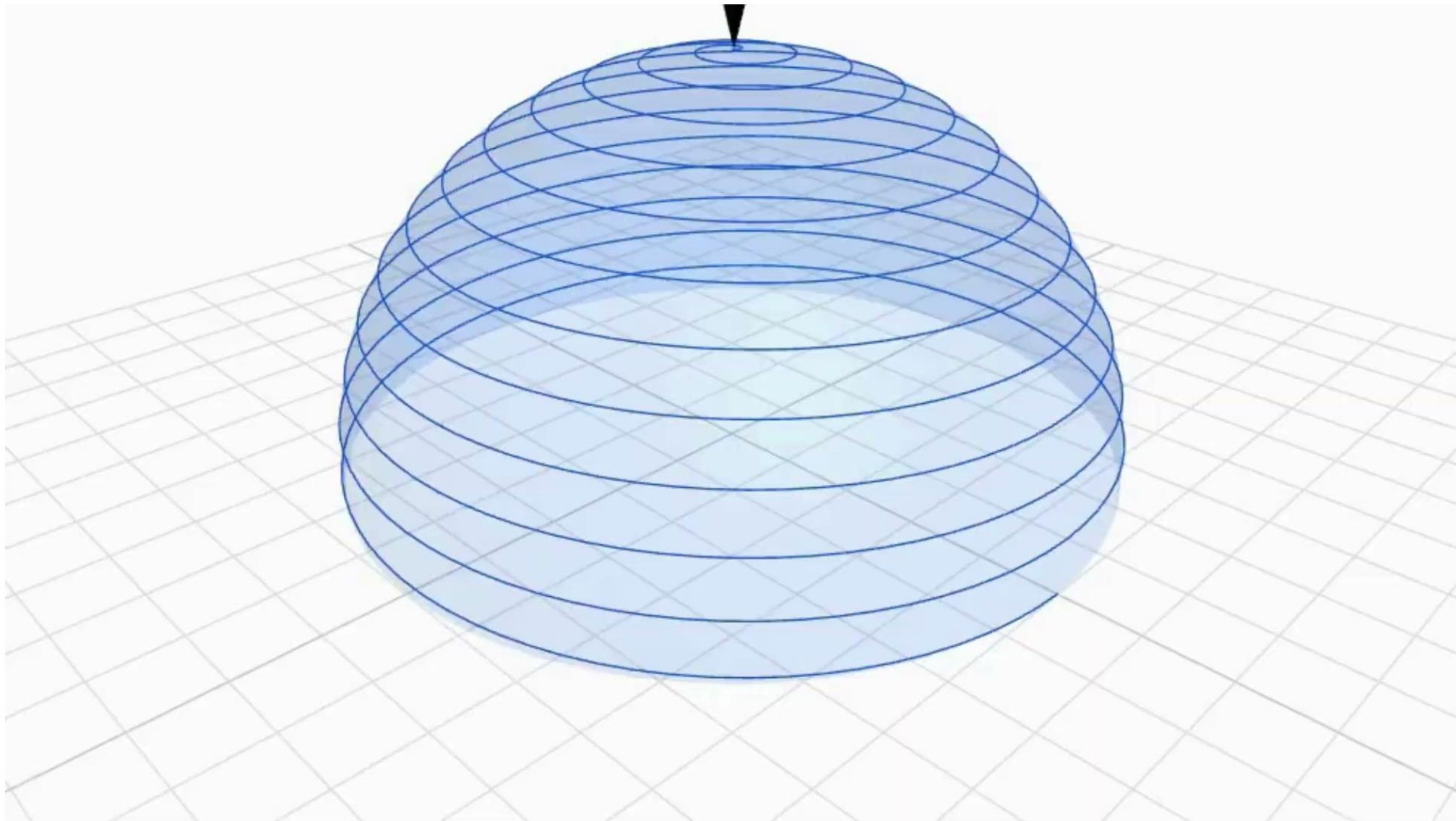
|                                                                                     | 순서 | 동작 방식   | 작업 설명                | 사용하는 좌표/명령                                 | 의미                         |
|-------------------------------------------------------------------------------------|----|---------|----------------------|--------------------------------------------|----------------------------|
|                                                                                     | 1  | movej   | 초기 좌표 복귀             | self.home_joint                            | 홈/초기 자세로 관절 이동             |
|                                                                                     | 2  | movej   | 도구 반납 접근             | self.tool_safe_joint                       | 도구 반납 위치 위쪽으로 관절 이동        |
|                                                                                     | 3  | movel   | 도구 최종 좌표 이동          | self.tool_place_posx                       | 도구 실제 위치로 직선 접근            |
|                                                                                     | 4  | gripper | 도구 내려놓기              | "open"                                     | 그리퍼를 열어서 도구 내려놓기           |
|    | 5  | movel   | 도구 안전 탈출             | self.tool_safe_posx                        | 도구 위치에서 위쪽으로 직선 탈출         |
|                                                                                     | 6  | movej   | 초기 위치 복귀             | self.home_joint                            | 다시 홈/초기 자세로 관절 이동          |
|    | 7  | movej   | 검사 장소 위 이동           | self.inspection_above_joint                | 검사 장소 위 좌표로 관절 이동          |
|                                                                                     | 8  | gripper | 그리퍼 열기               | "open"                                     | 물체를 잡기 전 그리퍼 열기            |
|                                                                                     | 9  | movel   | 물체 위치 하강             | self.inspection_place_posx                 | 검사 장소의 물체 위치로 직선 하강        |
|                                                                                     | 10 | gripper | 물체 잡기                | "close"                                    | 그리퍼를 닫아서 물체 잡기             |
|  | 11 | movel   | 검사 장소 위 상승           | self.inspection_above_posx                 | 물체를 잡은 뒤 검사 장소 위로 직선 상승    |
|                                                                                     | 12 | movej   | 분배 이동 (PASS 또는 FAIL) | self.pass_pos_joint 또는 self.fail_pos_joint | 검사 결과에 따라 합격/불합격 위치로 관절 이동 |
|                                                                                     | 13 | gripper | 물체 놓기                | "open"                                     | 분배 위치에서 물체 내려놓기            |
|                                                                                     | 14 | movej   | 최종 홈 복귀              | self.home_joint                            | 모든 작업 후 홈/초기 자세로 복귀        |

## 04 프로젝트 수행 경과 | 공정 후 분류 순서 및 동작 방식 플로우 차트



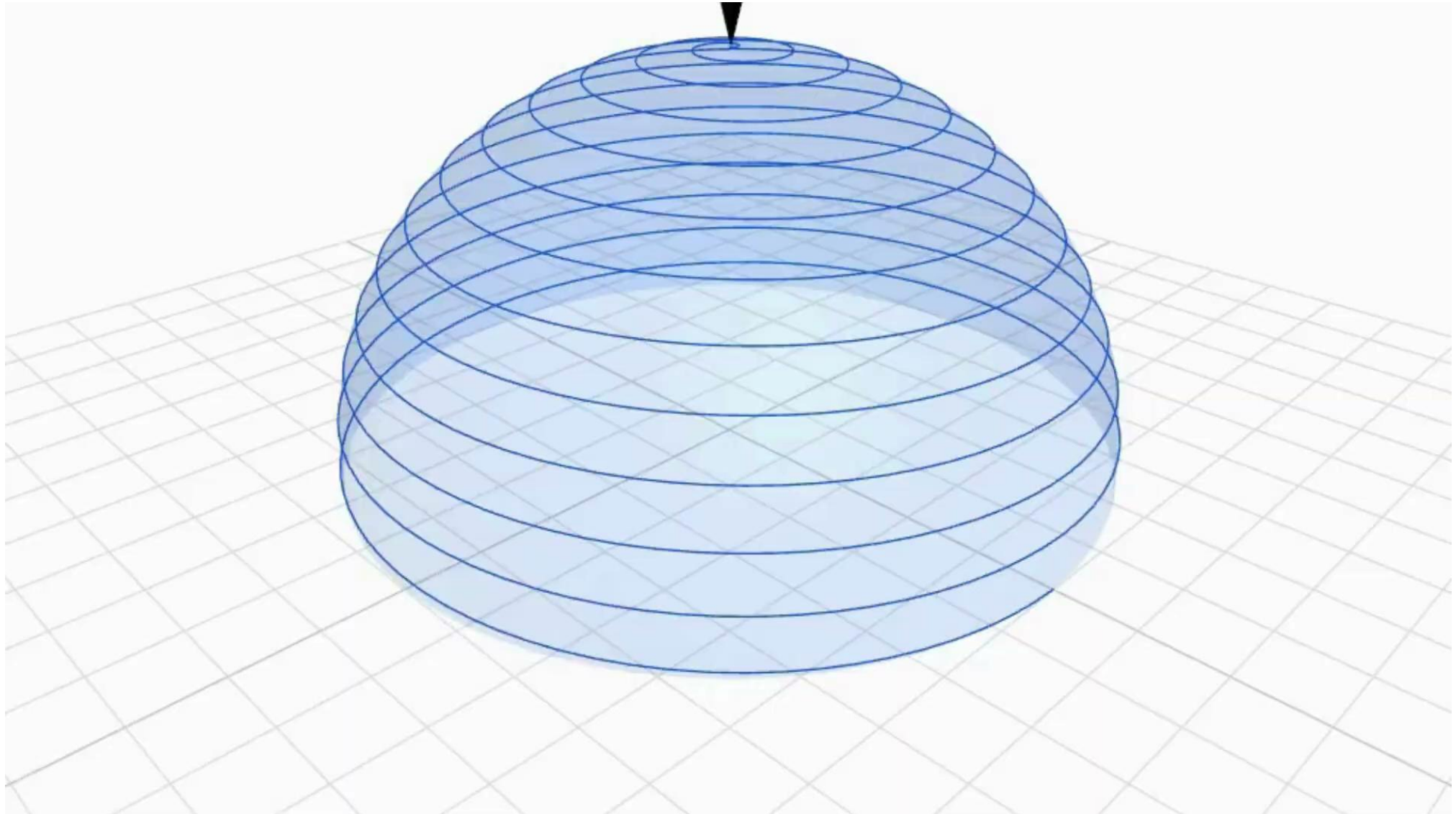
### 03 핵심 알고리즘 | scan\_moving 알고리즘

---



### 03 핵심 알고리즘 | scan\_moving 알고리즘

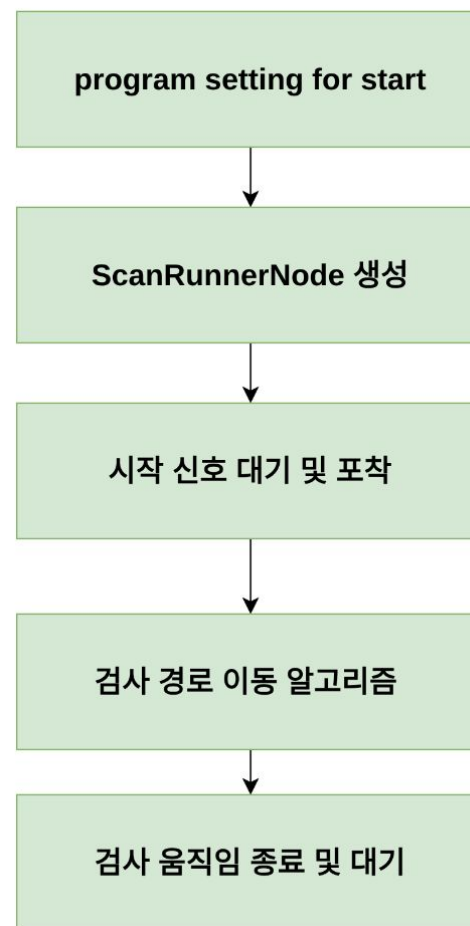
---



## 03 핵심 알고리즘 | scan\_moving 알고리즘

### scan\_moving 알고리즘

- 1. 프로그램 시작 세팅
- 2. ScanRunnerNode 생성
- 3. 시작 신호 대기 및 포착
- 4. 검사 경로 이동 알고리즘 시작
- 5. 시작 신호 대기 및 포착



## 03 핵심 알고리즘 | scan\_moving 알고리즘

### scan\_moving 알고리즘

#### · 1. 프로그램 시작 세팅

##### · 로봇 환경 초기화

DR\_init으로 제어로봇 ID, model 지정

##### · ROS2 통신망 접속

rclpy.init() 로컬 네트워크 접속

##### · 대기 상태 돌입

progress\_status에서 “02” 메시지를 받기 전까지 대기

#### program setting for start

로봇 환경 초기화  
DR\_init = ROBOT\_ID, ROBOT\_MODEL

ROS2 통신 접속  
rclpy.init()

대기 상태 돌입  
progress\_status에서 “02” 메시지를 받기 전까지 대기

## 03 핵심 알고리즘 | scan\_moving 알고리즘

### scan\_moving 알고리즘

## • 2. ScanRunnerNode 생성

### • publisher 생성

checking\_state (스캔중인 알려주는 메세지) 발행하는 publisher 생성

record\_enable (표면 접촉시 기록할지 안할지를 알려주는 메세지) publisher 생성

progress\_status scan (작업 완료시 “03” 메세지 발행) publisher 생성

### • subscriber 생성

progress\_status 토픽을 받는 subscriber 생성

### • progress\_callback

progress\_status에서 “02” 메세지를 받기 전까지 대기

### ScanRunnerNode 생성

#### publisher 생성

```
create_publisher(Bool, f'/{namespace}/checking_state', 10)
create_publisher(Bool, f'/{namespace}/record_enable', 10)
create_publisher(String, f'/{namespace}/progress_state', 10)
```

#### subscriber 생성

```
create_subscription
(String, f'/{namespace}/progress_status', self.progress_callback, 10)
```

## 03 핵심 알고리즘 | scan\_moving 알고리즘

### scan\_moving 알고리즘

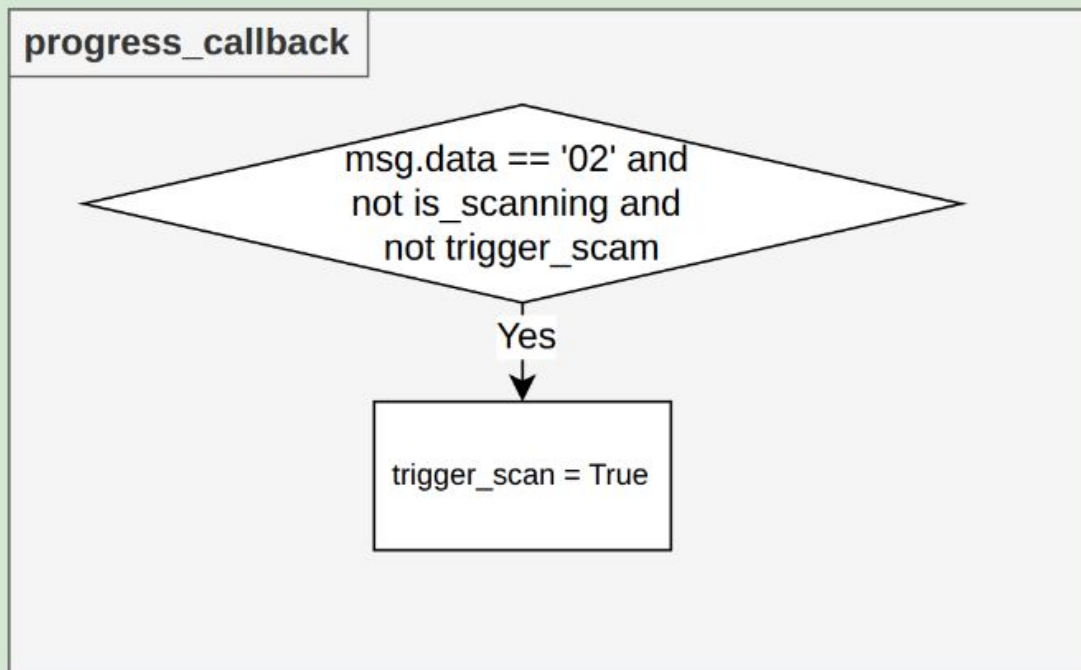
#### · 3. 신호 포착

#### · progress\_callback

progress\_status 토픽에 “02”라는 메시지를 잡아내면

trigger\_scan을 True 변환 스캔 시작

#### 신호 포착



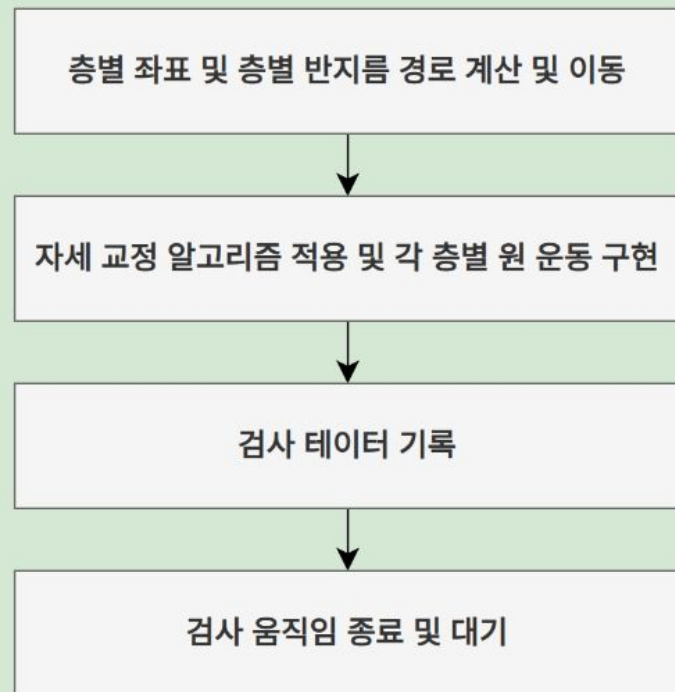
## 03 핵심 알고리즘 | scan\_moving 알고리즘

### scan\_moving 알고리즘

#### · 4. scan\_path\_move 알고리즘

- 1. 층별 좌표 및 층별 반지름 경로 계산 및 이동
- 2. 자세 교정 알고리즘 적용 및 각 층별 원 운동 구현
- 3. 검사 데이터 기록
- 4. 검사 움직임 종료 및 대기

#### 검사 경로 이동 알고리즘 시작



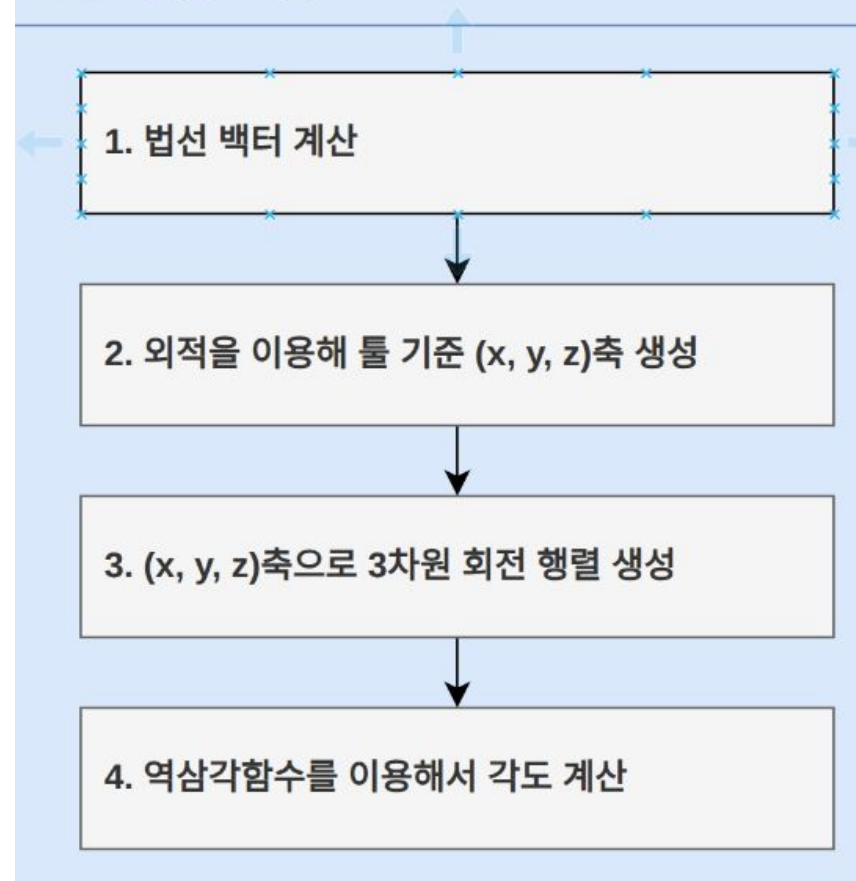
## 03 핵심 알고리즘 | scan\_moving 알고리즘

### scan\_moving 알고리즘

#### · 자세 교정 알고리즘

- 1. 법선 벡터 계산
- 2. 외적을 이용해 툴 기준 (x, y, z)축 생성
- 3. (x, y, z)축으로 3차원 회전 행렬 생성
- 4. 역삼각함수를 이용해 각도 계산

#### 자세 교정 알고리즘



## 03 핵심 알고리즘 | scan\_moving 알고리즘

### scan\_moving 알고리즘

#### · 자세 교정 알고리즘

##### · 1. 법선 벡터 계산

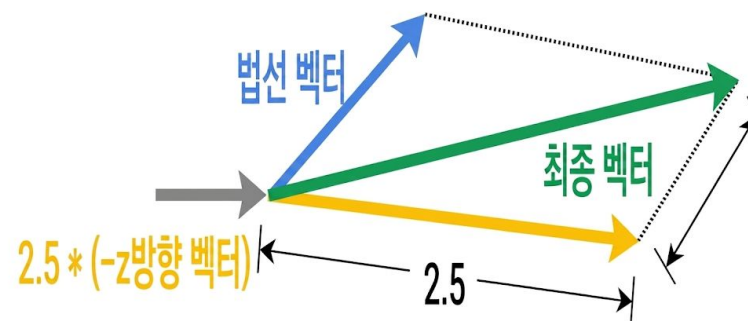
법선 벡터 = - (로봇팔의 위치(x,y,z) - 중심점의 위치(X, Y, Z)) (- 툴이 바라보는 방향)

##### · 문제점

법선 벡터만 사용시 로봇팔이 최하층 이동시 관절 한계각도에 도달

##### · 해결방안

최종 벡터 = 법선 벡터와 z축 벡터(-방향 벡터)를 1:2.5 비율로 조정



## 03 핵심 알고리즘 | scan\_moving 알고리즘

### scan\_moving 알고리즘

#### · 자세 교정 알고리즘

##### · 2. 외적을 이용해 툴 기준 (x, y, z)축 생성

최종 벡터(z축 벡터) 와 (0, 0, 1)벡터 (베이스 좌표계 기준) 외적 >> x축 벡터 생성

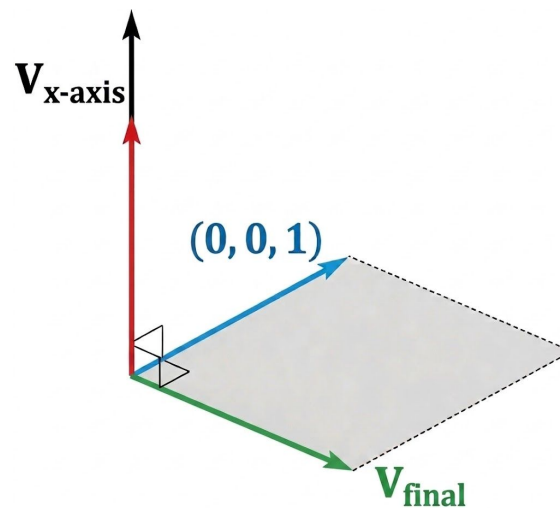
최종 벡터(z축 벡터) 와 (0, 0, 1) x축 벡터 외적 >> y축 벡터 생성

#### · 문제점

최종 벡터의 z축 값이 0.95 초과인 경우 (0, 0, 1) 벡터 외적시 결과값이 0이 되는 경우 발생

#### · 해결방안

최종 벡터의 z축 값이 0.95 초과인 경우 (0, 1, 0) 벡터 외적으로 각 축 벡터 계산



## 03 핵심 알고리즘 | scan\_moving 알고리즘

---

### scan\_moving 알고리즘

- 자세 교정 알고리즘

- 3. (x, y, z)축으로 3차원 회전 행렬 생성

x축 벡터, y축 벡터, 최종 벡터(z축 벡터)를 이용해 3차원 회전 행렬 생성

- 문제점

두산 M0609 모델은 회전형렬로 자세(orientation) 조정하지 않음

- 해결방안

회전형렬을 두산에서 사용하는 Z-Y-Z 오일러 각도로 변환

## 03 핵심 알고리즘 | scan\_moving 알고리즘

### scan\_moving 알고리즘

#### · 자세 교정 알고리즘

##### · 4. 역삼각함수를 이용해 각도 계산

3차원 회전 행렬과 **x축 벡터**, **y축 벡터**, **최종 벡터(z축 벡터)**를 이용해 3차원 회전 행렬 생성

$$A = \text{atan2}(R_{23}, R_{13}), B = \arccos(R_{33}), C = \text{atan2}(R_{32}, -R_{31})$$

$$R = \begin{bmatrix} R_{11} & R_{12} & \mathbf{R}_{13} \\ R_{21} & R_{22} & \mathbf{R}_{23} \\ \mathbf{R}_{31} & \mathbf{R}_{32} & \mathbf{R}_{33} \end{bmatrix}$$

##### · 문제점

B 값이 0에 가까우면 특이점 발생

##### · 해결방안

$$A = 0, B = \arccos(R_{33}), C = \text{atan2}(R_{21}, R_{11})$$

## 03 핵심 알고리즘 | scan\_moving 알고리즘

---

### scan\_moving 알고리즘

#### · 검사 경로 이동 알고리즘

##### · 1. 층별 좌표 및 층별 반지름 경로 계산 및 이동

반구의 중심과 반지름 계산으로 층을 나눔

삼각 함수를 사용해 각층 별 높이와 경로 원의 반지름 계산

move로 각 층 시작 지점으로 이동

##### · 문제점

1. 각 층 별 이동시 로봇팔의 움직임 부드럽게지 않음

2. 실제 물체 완벽한 구형 모양이 아님

##### · 해결방안

1. 정지 시간 제거 및 move에 블렌딩 ( $r = 15$ ) 값을 추가

2. 아래층으로 이동할 수 있도록 반지름이 최대 8.5mm까지 넓혀주는 비율 계산식 적용

## 03 핵심 알고리즘 | scan\_moving 알고리즘

### scan\_moving 알고리즘

#### · 검사 경로 이동 알고리즘

#### · 2. 자세 교정 및 각층별 원 운동 구현

자세 교정 알고리즘으로 자세 교정

movec(원주 구속 방식)으로 원 운동 구현

#### · 문제점

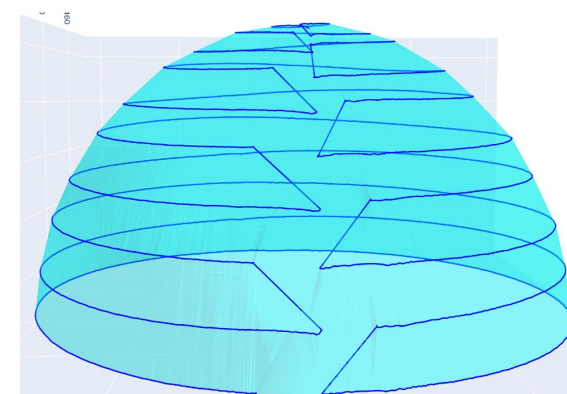
1. 6번 Joint의 회전 반경에 걸리는 문제 발생

2. 각층을 내려갈때마다 6번 Joint 이전 회전 운동인한 각도 누적으로 회전반경에 걸리는 문제 발생

#### · 해결방안

1. movec 움직임전에 6번 Joint를 -180로 설정하고 회전반경을 -180 ~ 180로 설정

2. 홀수층 시계방향 짝수층 반시계방향으로 돌수 있도록 로직 설정



## 03 핵심 알고리즘 | scan\_moving 알고리즘

### scan\_moving 알고리즘

- 검사 경로 이동 알고리즘

- 2. 자세 교정 및 각층별 원 운동 구현

각층 별로 계산한 시작점과 반지름 값을 이용해 원 운동

- 문제점

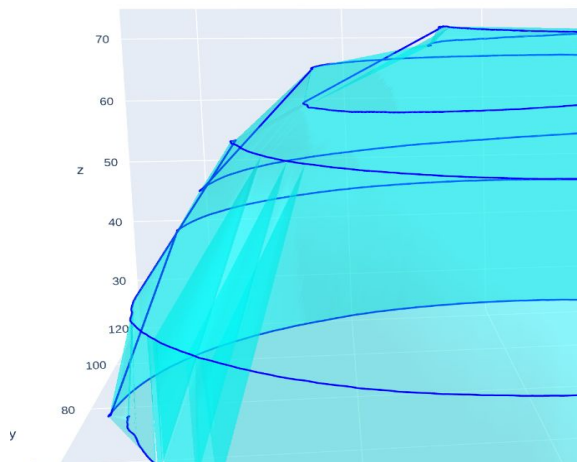
이론상의 좌표와 실제 물체의 좌표가 1~2mm만 다른 경우

로봇이 강하게 물체를 찍거나 허공을 움직이는 상황 발생

- 해결방안

순응제어 적용( $\text{stiff} = [1, 3000, 3000, 300, 1, 3]$ )

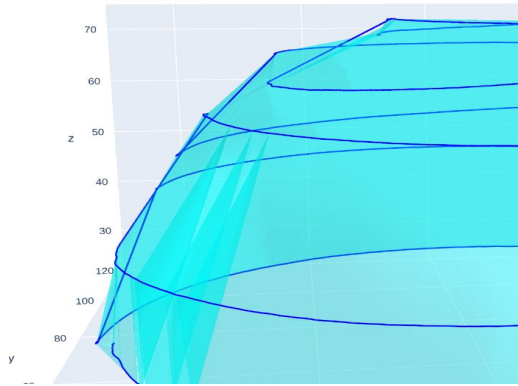
x축 강성을 과 y축 기준 회전 강성 극도로 낮춰서 돔 표면에 따라 갈 수 있도록 설정



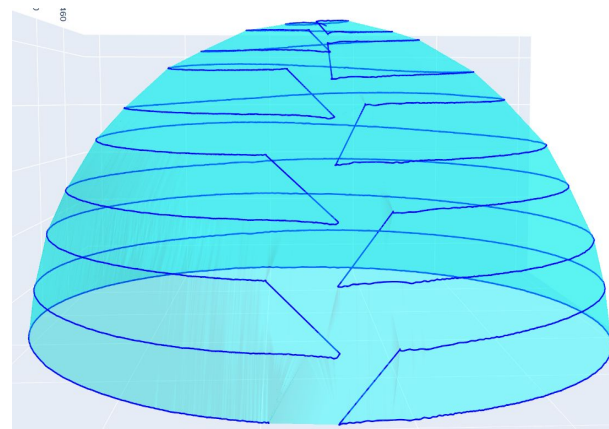
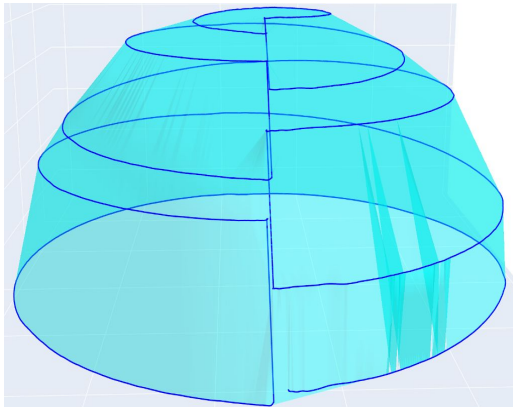
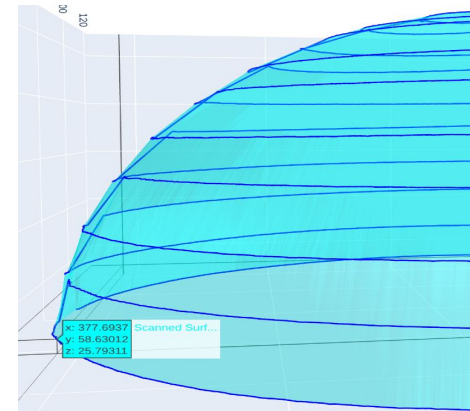
## 03 핵심 알고리즘 | scan\_moving 알고리즘

### scan\_moving 알고리즘

순응제어 적용 전



순응제어 적용 후



## 03 핵심 알고리즘 | scan\_moving 알고리즘

### scan\_moving 알고리즘

#### · 검사 경로 이동 알고리즘

#### · 3. 검사 데이터 기록

원 운동동안 툴 위치 값을 기록

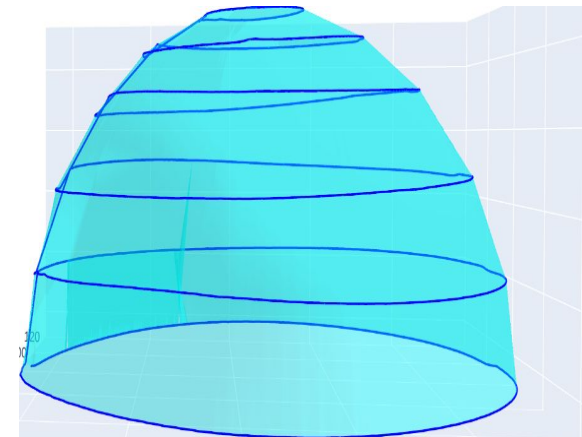
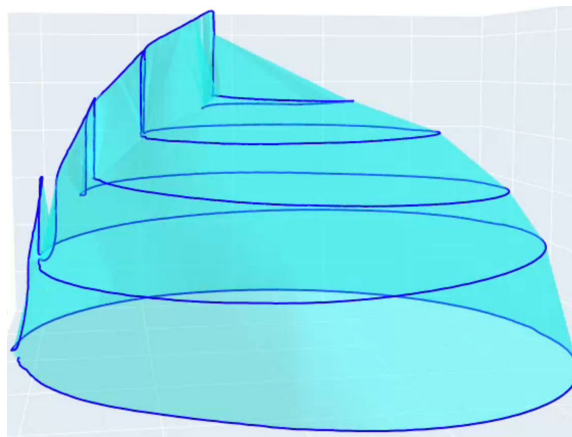
#### · 문제점

물체의 표면에 접촉하지 않아도 데이터를 기록하는 문제 발생

#### · 해결방안

공중에서 이동시 기록 off

표면에 접촉하고 movec움직임 직전에 기록 on



## 03 핵심 알고리즘 | scan\_moving 알고리즘

---

### scan\_moving 알고리즘

- 검사 경로 이동 알고리즘
- 4. 검사 움직임 종료 및 대기
- 전체 검사 사이클 종료 및 Ready 대기 상태 복귀

전체 검사 사이클 종료 및 Ready 대기 상태 복귀

## 03 핵심 알고리즘 | scan\_moving 알고리즘

---

### scan\_moving 알고리즘

- **실제 데이터 기준 최적화**

- **문제점**

표면 추적만으로 실제 표면을 누를 수 없음

- **해결방안**

오프셋을 사용해 목표 궤적보다 실제 2~4mm 정도를 더 누르면서 이동

# 04 프로젝트 수행 경과 | 표면 검사 알고리즘

## 01. Input (ROS 2)

**/joint\_states**  
관절 상태 수신

**Torque Service**  
외부 토크 데이터 요청

**State Monitor**  
기록 상태 감시

## 02. Processing & Analysis Engine

**Forward Kinematics**  
Joint Angle → XYZ 좌표

**Spike Detection**  
이상 토크 감지 및 필터링

**Data Buffer**  
분석 데이터 일시 저장

**Cubic Spline**  
궤적 복원 및 보간

**Error Calculation**  
최대 편차 및 오차율 산출

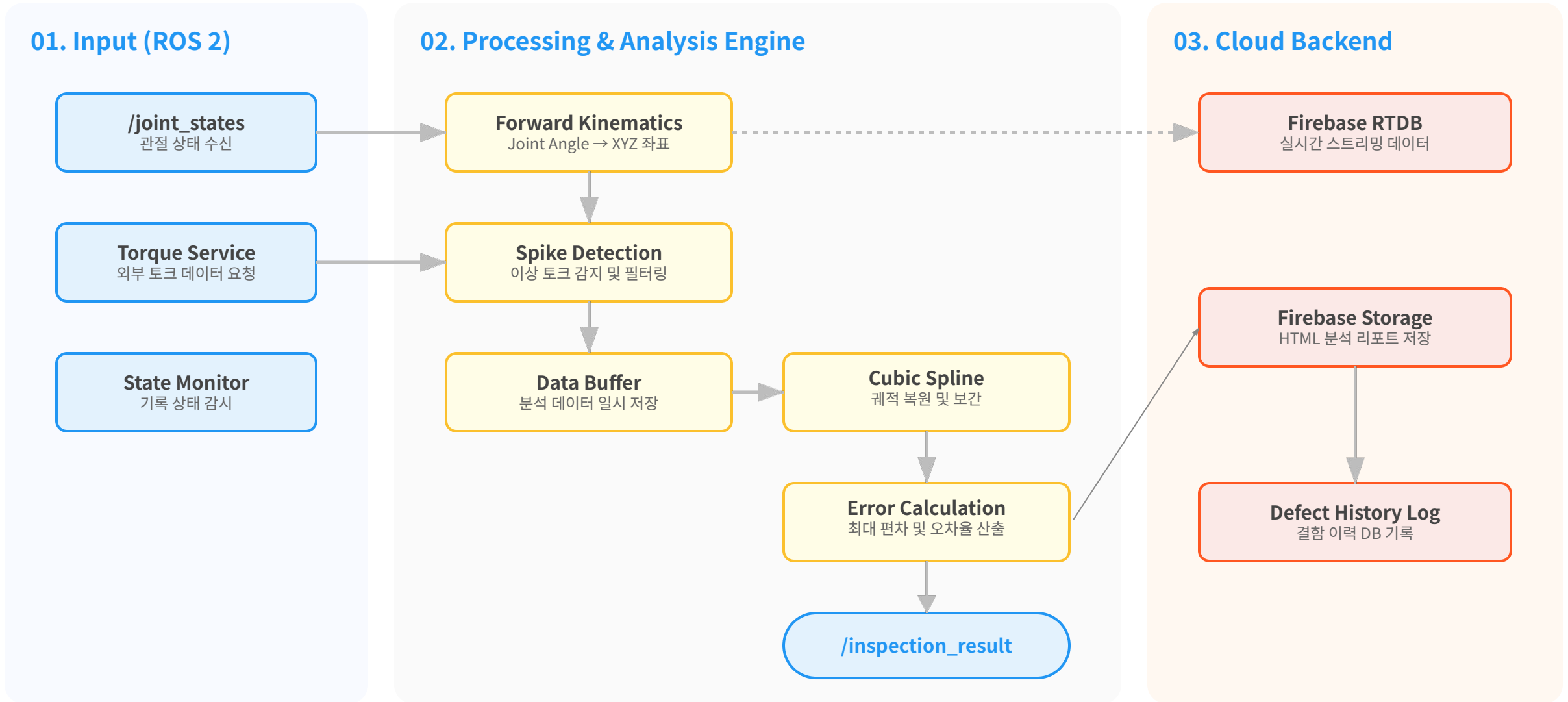
**/inspection\_result**

## 03. Cloud Backend

**Firebase RTDB**  
실시간 스트리밍 데이터

**Firebase Storage**  
HTML 분석 리포트 저장

**Defect History Log**  
결함 이력 DB 기록



# 04 프로젝트 수행 경과 | 표면 검사 알고리즘

## 01. 정밀 실시간 연산 (FK)

### 실시간 정기구학(Forward Kinematics) 연산

관절 각도 데이터를 4x4 변환 행렬로 처리하여 50ms 주기로 TCP 좌표를 정밀 산출합니다.

## 02. 지능형 결함 탐지

### 적응형 토크 Spike 감지

외부 토크의 이동 평균(Baseline) 대비 급격한 변화를 탐지하여 결함 및 충돌을 실시간 판단합니다.

## 03. 궤적 복원 & 검증

### Cubic Spline 궤적 분석

이상 구간의 정상 궤적을 수학적으로 복원(Interpolation)하고 예측값과의 편차(L2 Norm)를 통해 검증합니다.

## 04. 데이터 클라우드

### 클라우드 데이터 파이프라인

Firebase 및 Storage 연동을 통한 실시간 데이터 동기화 및 3D 분석 리포트를 자동 생성합니다.

# 04 프로젝트 수행 경과 | 표면 검사 알고리즘

고정밀 실시간 정기구학(Forward Kinematics) 엔진

## 개념 및 역할

- 로봇 관절 회전각( $q_1 \sim q_6$ )을 입력받아 베이스 기준 TCP( $x, y, z$ ) 위치 산출
- 토크 이상 감지 시 정확한 공간 좌표를 파악하여 결함 위치를 특정하는 핵심 소스

## DH 파라미터 적용

두산 로봇(DSR) 물리 규격 반영:

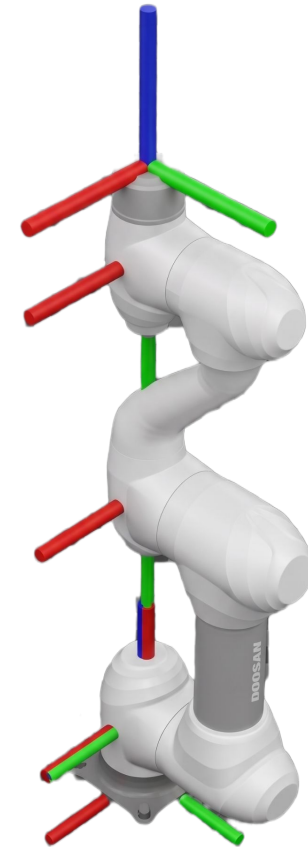
- Base-Shoulder: z-offset 0.1345m
- Arm Segments: 0.411m, 0.368m
- Tool Offset: 최종 보정 0.2586m

## 동차 변환 행렬 연산 (Chain Multiplication)

$$T_i^{i-1} = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{0\_tool} = T_{01}(q_1) \cdot T_{12}(q_2) \cdot T_{23}(q_3) \cdot T_{34}(q_4) \cdot T_{45}(q_5) \cdot T_{56}(q_6) \cdot T_{tool}$$

각 링크 간 관계를 4x4 행렬로 정의하여 연쇄 연산 수행



# 04 프로젝트 수행 경과 | 표면 검사 알고리즘

M0609 실시간 토크 데이터 분석 및 적응형 이상 탐지

## 데이터 획득 및 전처리 (Data Acquisition)

- **하드웨어 인터페이스:** 두산 로보틱스 M0609 관절 내장 토크 센서 활용
- **데이터 소스:** dsr\_msgs2 서비스 GetExternalTorque를 통한  $\tau_{ext}$  수신
- **샘플링 주행:** 20Hz(0.05초) 주기의 전 관절(J1~J6) 실시간 모니터링

## 이상 감지 알고리즘 (Detection Logic)

- **적응형 베이스라인:** 최근 5프레임 이동 평균(Moving Average) 기반 기준점 설정
- **임계치 판정:** 최대 편차(max\_spike) **1.7 Nm** 초과 시 즉각 판단
- **진입 구간 안정화:** 초기 1.0초(20프레임) 관성 노이즈 감지 제외 로직 적용



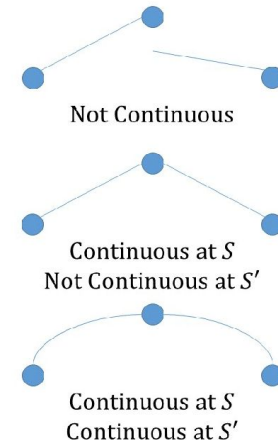
# 04 프로젝트 수행 경과 | 표면 검사 알고리즘

## Cubic Spline 보간 기반의 궤적 복원 및 정밀 오차 분석

### 궤적 복원 알고리즘 (Path Reconstruction)

- 데이터 윈도우 구성: 토크 스파이크 발생 '결함 시작점' 기준 전후방 일정 구간(WINDOW\_FRAMES) 설정
- 정상 데이터 샘플링: 결함 구간을 제외한 이전/이후 정상 주행 좌표만을 보간법 입력값으로 사용
- Cubic Spline Interpolation: 3차 다항식 곡선으로 '이상적(Ideal) 궤적' 산출

### Cubic Splines



### 수치적 오차 계산 (Quantitative Analysis)

#### 이탈 거리 산출 (Euclidean Distance):

- $$e = \sqrt{(x_a - x_i)^2 + (y_a - y_i)^2 + (z_a - z_i)^2}$$
- 정량적 품질 판정 (Error Rate %): 최대 편차( $e_{max}$ )를 허용치(5mm)와 비교
- FAIL 판정: 오차율 5.0% 초과 시 자동 리포트 생성

### 알고리즘 안정성 (Numerical Stability)

- 발산 방지 로직: 스플라인 연산 중 수치적 발산 실시간 감지
- 물리적 한계 필터링: 10mm 이상의 불가능한 변위 발생 시 보정치 즉각 적용
- 시스템 신뢰성: 노이즈 및 특이값에 의한 오판 방지로 안정적 제어 루프 통합

# 04 프로젝트 수행 경과 | 정량적 평가 지표

## Cubic Spline 보간 기반의 궤적 복원 및 정밀 오차 분석

### 1. 데이터 수집 및 구간 분리 (Windowing)

먼저 로봇의 고장이 발생했을 때 발생하는 특정한 토크 증분 신호가 감지되는 작업 구간을 선별합니다.

- 트리거 조건 설정:** 위치 편차(Position) 보다 토크(Torque) 데이터의 미분값(변화량)이 크게 발생하는 시점을 탐지합니다.
- 검사 윈도우(Windowing):** 탐지된 시점 전후로 일정 시간 동안의 데이터 흐름을 분석 창으로 잡습니다.
- Window 구성:**  $[t_s - \Delta t, t_s + \Delta t]$  범위의 정상 궤적( $W_{normal}$ )과 실제 측정 궤적( $W_{measured}$ )을 추출합니다.

### 2. 이상적인 경로(Ideal Path) 생성

작업 표면이 비정상적이기 때문에 단순히 앞뒤 좌표를 직선으로 연결하면 표면 곡률 때문에 오차가 생깁니다.

- 곡선 근사화:** 정상 구간의 점 데이터(X, Y, Z)를 사용하여 3차 다항식 곡선(Polynomial)으로 실제 모델을 복제합니다.
- 구현 방법:** 최소자승법(Least Squares Method)을 사용하여 가장 매끄러운 궤적(Cubic Spline)을 수학적으로 추론합니다.

### 3. 오차 판단 및 정량적 검증 지표

이상적인 경로( $W_{ideal}$ )와 실제 로봇의 측정값( $W_{measured}$ )을 비교하여 결함(오차)을 계측합니다.

#### ● 최대 편차 (Maximum Deviation)

전체 분석 구간 중 가장 크게 벌어진 거리입니다. 균열의 깊이나 폭을 직관적으로 대변합니다.

$$Error_{max} = \max | P_{measured}(t) - P_{ideal}(t) |$$

#### ● 평균 제곱 오차 (RMSE)

분석 구간 전체에서 로봇의 궤적이 얼마나 일정한지를 나타내는 종합적인 크기를 나타냅니다.

#### ● 단차 적분 오차 (Integral of Error)

단순 좌표의 차이뿐만 아니라 오차 면적의 실제 결과가 3D 상의 면적 오차를 나타내어 균열의 깊이를 측정합니다.

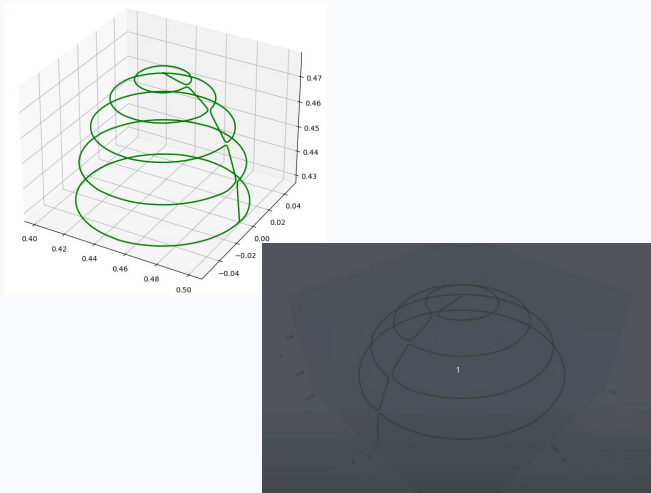
# 04 프로젝트 수행 경과 | 표면 검사 알고리즘

좌표 기반 검사 문제 해결 과정

## Phase 01

### 정지된 2D의 한계 (v00~v02)

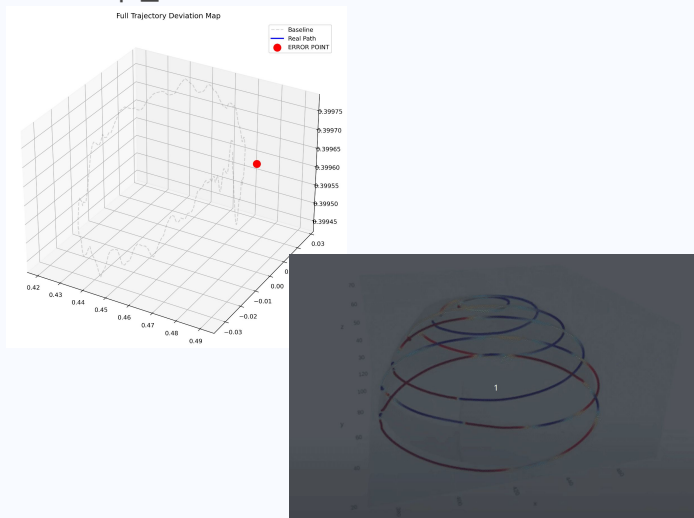
- **문제:** matplotlib 기반 PNG 저장. 3차원 움직임을 평면으로만 확인 가능하여 다각도 검토 및 오차 방향성 파악 불가
- **해결:** Plotly.js 기반 인터랙티브 HTML 렌더링 도입. 마우스 휠 줌 및 자유 회전 프레임워크 구축



## Phase 02

### '점 vs 점' 거리 연산의 오류 (v03~v11)

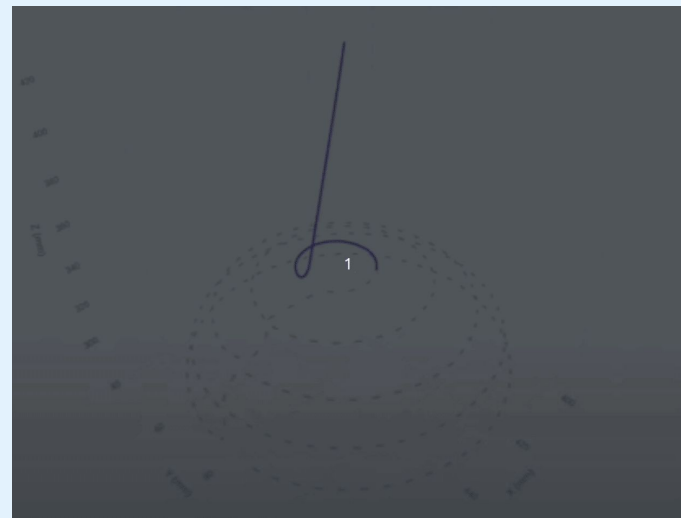
- **문제:** **ckdTree** 기반 근접 점 측정. 샘플링 점 사이의 빈 공간을 이탈로 오판하여 가짜 불량 (False Positive) 발생
- **해결:** 수학적 면(Mesh) 생성 및 trimesh 기반 수직 거리 계산으로 전환. 완벽한 곡면 추종 검사 구현



## Phase 03

### 웹 연동 및 동기화 혁신 (v12~v14)

- **문제:** 로컬 전용 리포트로 현장 작업자의 즉각적 확인 불가 및 로봇-웹 간 시간차 발생
- **해결:** Firebase Admin SDK 결합 및 클라우드 즉시 업로드. **now.timestamp()** 기반 ms 단위 정밀 동기화



# 04 프로젝트 수행 경과 | 표면 검사 알고리즘

좌표 기반 검사의 한계점

## 01. 순응 제어의 역설 (Compliance Masking)

로봇의 유연한 대응이 결함 데이터를 가리는 현상

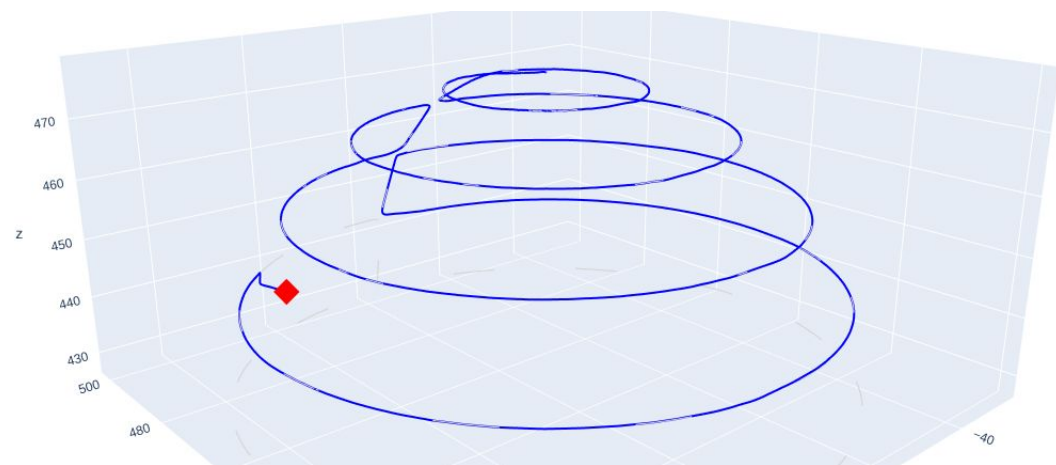
- **현상:** task\_compliance\_ctrl 설정으로 외부 충격 흡수
- **문제:** 미세 크랙 통과 시 로봇 팔이 충격을 흡수하며 경로 유지
- **결과:** 좌표 변화 거의 없음 (오차 1~2mm 내외)이나 실결함 존재



## 02. 해상도 부족 (Resolution Gap)

좌표 데이터만으로는 감지 불가능한 미세 결함

- **허용 오차:** 3.0mm는 '위치 이탈' 감지에는 적절함
- **제약:** 머리카락 굵기나 얇은 틈 감지에는 좌표 데이터가 둔탁함
- **결과:** 정밀한 크랙 판단을 위한 추가 데이터(토크 등) 필요



# 04 프로젝트 수행 경과 | 파라미터 최적화: 미세 결함 탐지를 위한 임계점 실험

Trial & Error 과정을 통한 감지 정밀도 및 기계적 안전성 확보

## 실험 설계

### 핵심 실험 변수 제어

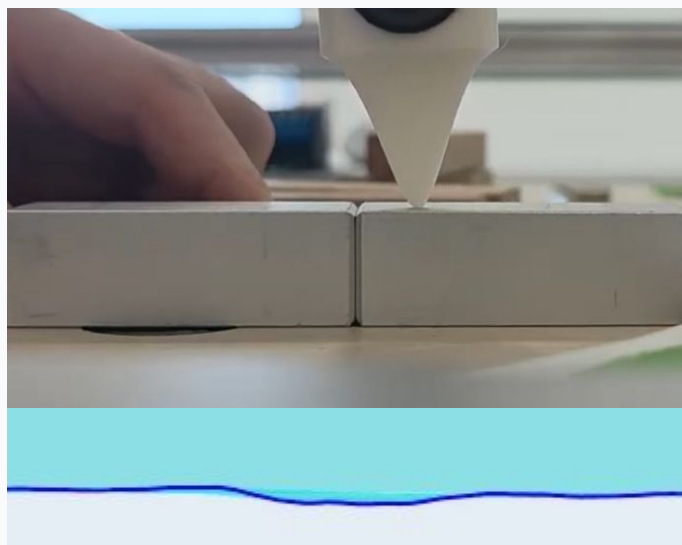
로봇 툴이 바닥에 닿는 깊이와 누름 강도를 결정하는 핵심 파라미터를 정밀하게 조정

- `target_z` (바닥 감지 높이)
- `zero_z_offset` (툴 누름 오프셋)

### 실험 범위

- 표면 높이 오프셋 구간: `-2mm ~ -6mm`
- 누르는 힘 오프셋 구간: `-4mm ~ -10mm`
- 테스트 타겟: `2mm` 및 `4mm` 미세 크랙

## 데이터 가시화



### 임계점 분석 (Threshold Analysis)

각 오프셋 단계별 측정되는 좌표값을 분석을 통해 결함 부위 통과시 만들어지는 그래프로 감지 가능 여부 판단함

## 최적 도출 결과

### ✓ 최적 오프셋 구간 확보

기계적 부하와 탐지 감도의 균형점 도출

- **부하 최소화:** 로봇 관절에 가해지는 과도한 토크 방지
- **정밀도 극대화:** `2mm` 이하 미세 결함의 명확한 인지 가능
- **신뢰성 확보:** 반복 테스트를 통한 데이터 재현성 검증 완료

# 04 프로젝트 수행 경과 | 실험적 검증: 최적 파라미터 도출 및 PoC 성공

물리적 결함 탐지 정밀 제어 및 최적 임계점 도출 결과 보고

## 핵심 실험 결과

### 최적의 탐지 파라미터 조합

툴 무게 보정값을 -6으로 설정하고, 측정된 바닥 높이에서 -4mm의 추가 오프셋을 적용했을 때 2mm와 4mm의 미세 틈새를 가장 정확하게 인지함

**Offset: -4.0mm**

**Weight: -6.0**

탐지 정밀도와 기계적 안전성의 완벽한 균형

틈 길이: 2mm | 툴 무게: -6 | 바닥 오프셋: -2mm



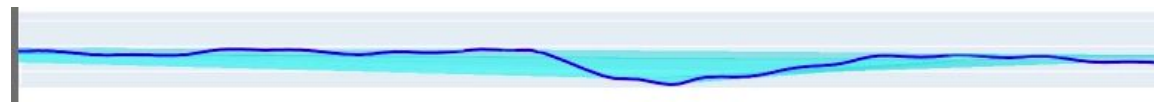
틈 길이: 2mm | 툴 무게: -8 | 바닥 오프셋: -2mm



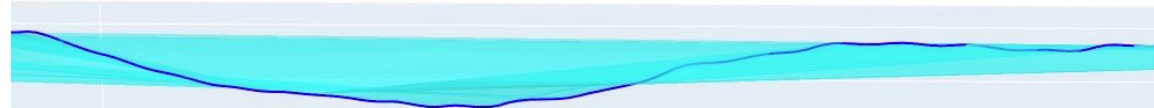
틈 길이: 2mm | 툴 무게: -10 | 바닥 오프셋: -2mm



틈 길이: 2mm | 툴 무게: -6 | 바닥 오프셋: -4mm



틈 길이: 4mm | 툴 무게: -6 | 바닥 오프셋: -4mm



# 04 프로젝트 수행 경과 | 위치 제어에서 역학 제어로

구조적 한계를 극복하기 위한 데이터 소스의 질적 전환

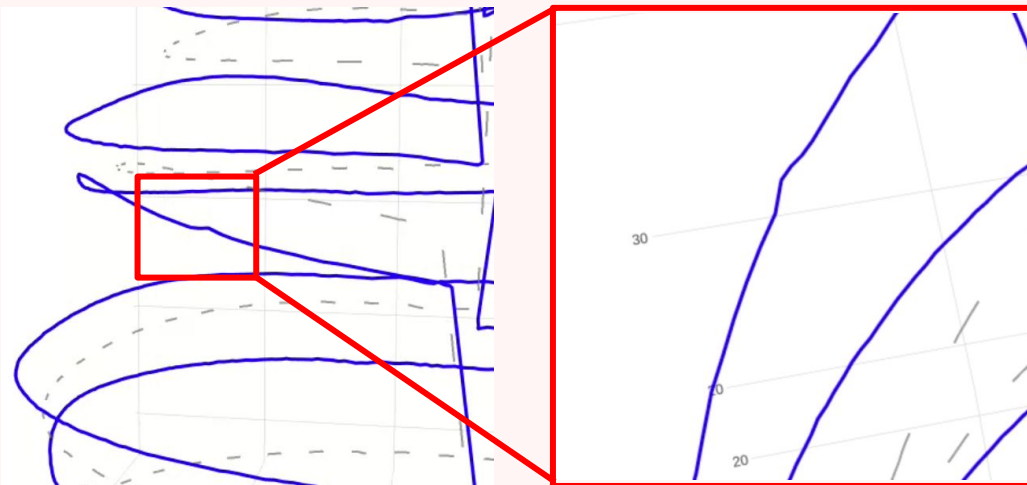
## 기존 방식의 한계

### 곡면 구조에서의 궤적 왜곡

평면에서 성공한 **고정 Z-오프셋 방식**을 원형 및 곡면 구조에 적용하려 했으나, 표면 각도 변화에 따른 수직 항력 불균형으로 궤적 왜곡 발생

#### Critical Issues:

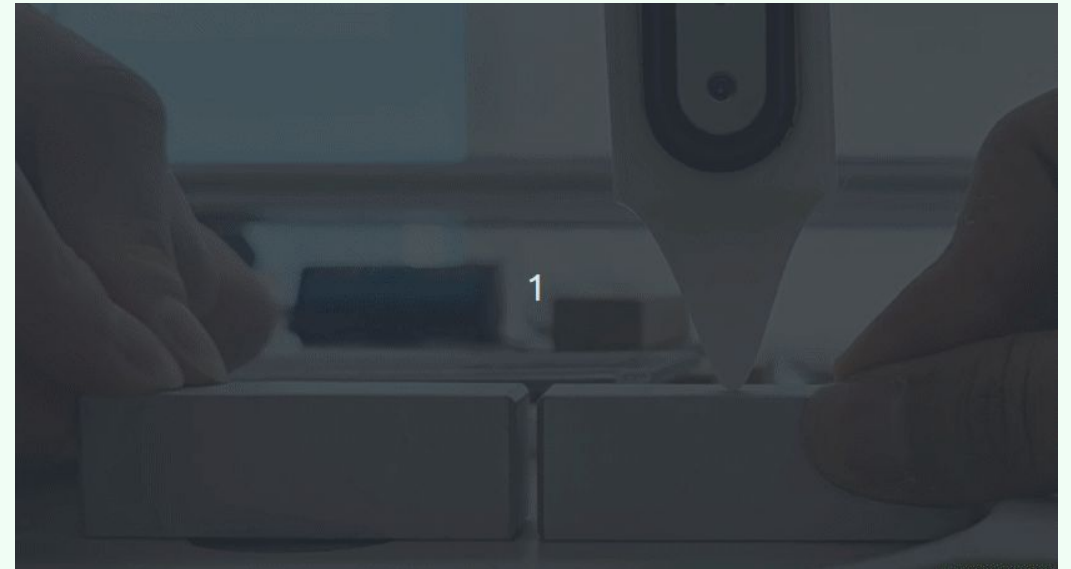
- 곡률에 따른 유효 압력 변화
- 표면 마찰력 변화에 의한 노이즈
- 단순 위치 기반 제어의 한계점 봉착



## 차세대 감지 전략

### 토크 센서 기반 역학 제어

결합을 지날 때 로봇 팔에 가해지는 **물리적 충격(Impact)**을 포착하여, 각 조인트의 토크(Torque) 데이터를 활용하는 지능형 감지 방식으로 전환



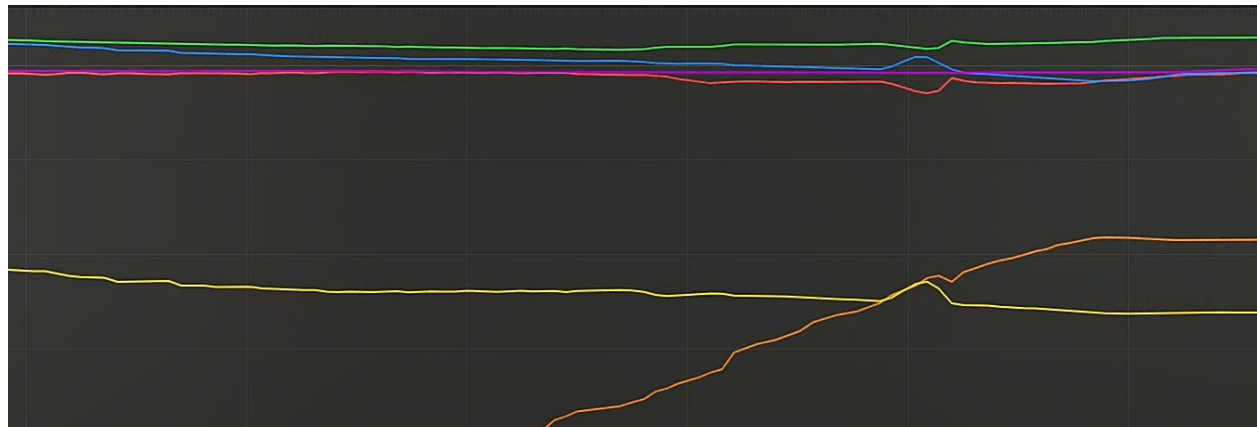
# 04 프로젝트 수행 경과 | 토크 기반 정밀 검출 알고리즘의 문제 해결

물리 데이터 필터링에서 3D 공간 불량 지도 구축까지

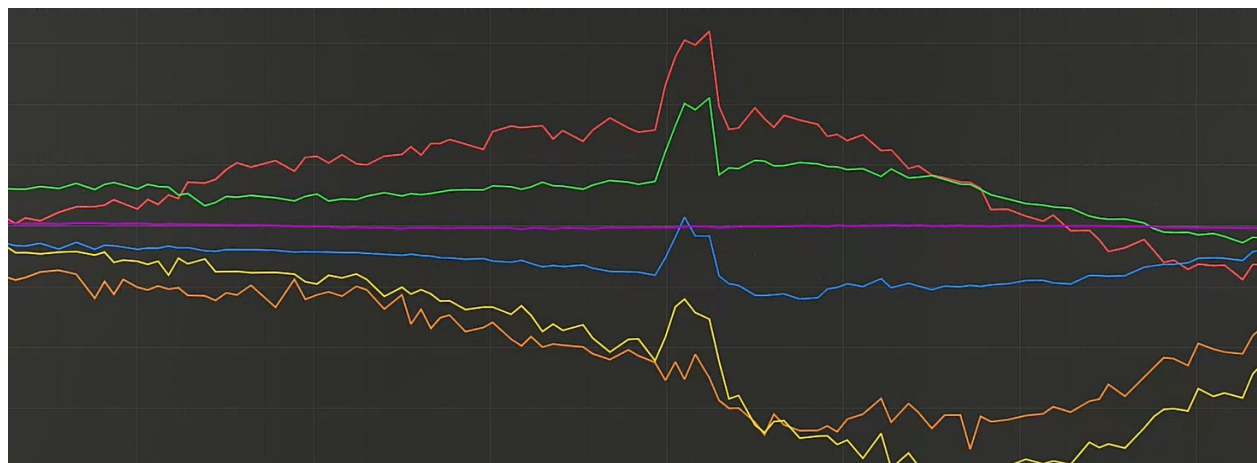
## Phase 01

### 데이터의 근원 탐색 (v00~v01)

- **문제:** GetJointTorque 사용 시 중력 및 관성력이 포함되어 순수 외력 추출이 어려움
  - **해결:** **GetExternalTorque** 전환. 자체 하중을 계산 제외한 순수 Tool 외력(Nm) 확보
- 
- **문제:** 불량 감지 임계값 설정이 너무 낮으면 모든 노이즈를 불량으로 감지
  - **해결:** **1.6~1.7** 에서 가장 안정적인 불량 감지가 가능했음



GetJointTorque



GetExternalTorque

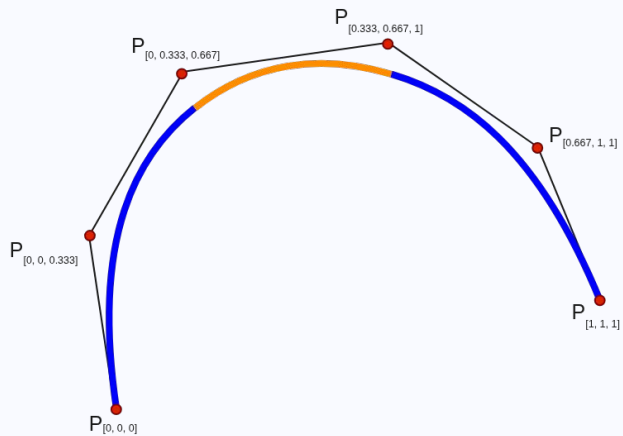
# 04 프로젝트 수행 경과 | 토크 기반 정밀 검출 알고리즘의 문제 해결

물리 데이터 필터링에서 3D 공간 불량 지도 구축까지

## Phase 02

### 크랙 판정 알고리즘 고도화 (v02~v04)

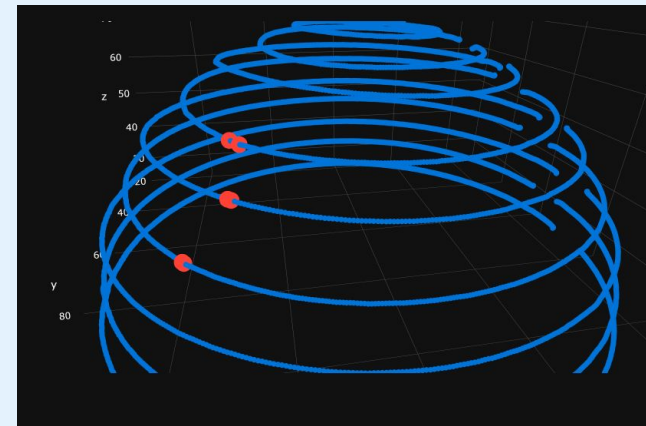
- **문제:** 방향 전환 시 발생하는 관성 노이즈와 실제 크랙 충격 간의 구분이 모호함
- **해결:** **Detection Window** 및 CubicSpline 보간법 도입. 토크 변화율(Slope) 기반 이상치 분리



## Phase 03

### 공간 정보와의 재결합 (v05~v08)

- **문제:** 크랙 존재 여부는 인지하나 구체적인 발생 위치(X, Y, Z) 특정 불가
- **해결:** **Forward Kinematics** 연동. 찰나의 관절 각도를 3D 좌표로 변환하여 '불량 지도' 시각화
- **결과:** 정상(파란 점) vs 크랙(빨간 점) 3D 매핑



# 04 프로젝트 수행 경과 | 모니터링 방식별 정밀 비교 분석

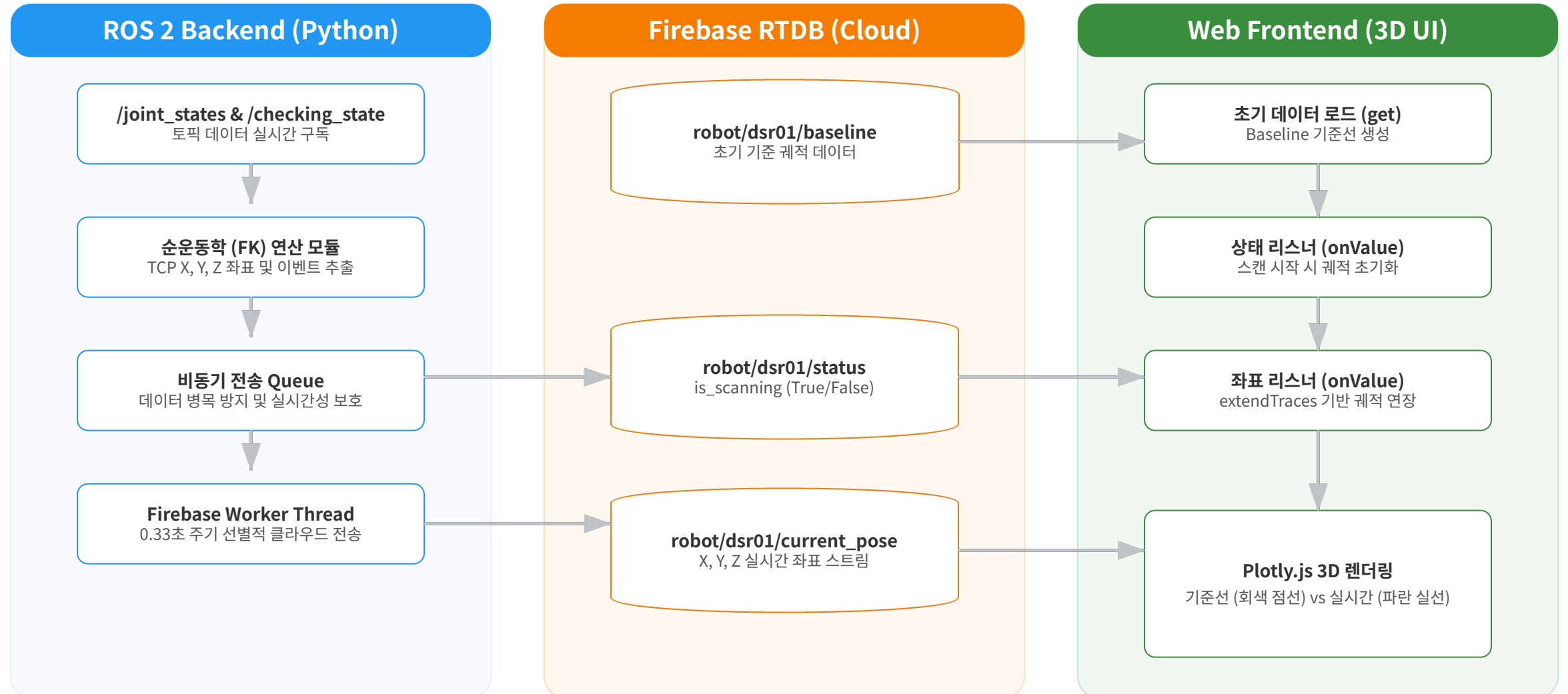
좌표 기반 거시적 탐지와 토크 기반 미시적 진단의 상호 보완 체계

| 구분     | 좌표 기반 (monitoring_14)          | 토크 기반 (torque_monitor_08)        |
|--------|--------------------------------|----------------------------------|
| 핵심 데이터 | TCP 3D 좌표 (X, Y, Z)            | 외부 조인트 토크 (External Torque)      |
| 감지 대상  | 경로 이탈, 물리적 충돌                  | 미세 크랙, 표면 거칠기, 마찰 이상             |
| 수학 모델  | Delaunay Mesh / Point-to-Plane | Cubic Spline / Torque Derivative |
| 강점     | 거시적인 형태 변형 감지                  | 미세한 물리적 저항 변화 포착                 |



# 04 프로젝트 수행 경과 | 실시간 디지털 트윈 아키텍처 및 데이터 흐름

ROS 2 백엔드와 Firebase 클라우드, Plotly.js 웹 UI 간의 유기적 데이터 연동 체계



# 04 프로젝트 수행 경과 | 디지털 트윈 기반 실시간 3D 모니터링 시스템

비동기 클라우드 브릿지 및 웹 기반 3차원 궤적 시각화 대시보드

## 통신 브릿지 (Python - Firebase Bridge)

- **비동기 큐(Queue) 아키텍처:** 로봇 제어기(ROS 2)의 실시간성 보호를 위해 송신 Thread와 연산 작업을 분리, **통신 지연 0%** 달성
- **지능형 데이터 샘플링:** 트래픽 최적화를 위해 0.33초(3Hz) 주기로 필수 위치 데이터(X, Y, Z)만 선별 스트리밍
- **상태 자동 동기화:** 검사 시작/종료 이벤트 감지 시 클라우드 상태 값 (is\_scanning) 자동 업데이트

## 웹 3D 렌더링 엔진 (HTML/JS - Digital Twin UI)

- **Plotly.js 기반 3D 시각화:** 무설치 웹 구동, 실제 공간과 동일한 1:1 비율(Cube Aspect Mode)의 정밀 환경 제공
- **직관적 궤적 비교:** Baseline(회색 점선) 대비 **Actual Path(파란 실선)**를 중첩 렌더링하여 공정 이탈 즉시 확인
- **이벤트 반응형 UI:** Firebase onValue 리스너 기반 실시간 그래프 확장 및 검사 시작 시 자동 초기화(Auto-Clear)

# 04 프로젝트 수행 경과 | Firebase 비동기 데이터 브릿지 및 실시간 스트리밍

멀티스레딩 기반의 비압착식 데이터 전송 및 클라우드 동기화 기술

## 01. 데이터 소스 및 좌표 생성

- **원천 데이터:** `/joint_states` 토픽의 6축 실시간 각도 수집
- **좌표 산출:** 순운동학(FK) 모듈을 통한 TCP x, y, z (mm) 좌표 생성
- **트리거 연동:** `/checking_state` 신호 기반 전송 자동 제어

## 02. 비동기 통신 로직

- **워커 스레드 분리:** Threading/Queue 활용, 제어-통신 루프 완전 분리
- **Latency-Free:** 네트워크 대기 시간이 ROS 2 실시간성에 영향 제로
- **데이터 흐름:** Callback → Queue.put → Worker Thread (Firebase)

## 03. 성능 및 최적화

- **지능형 샘플링:** 3Hz(0.033s) 주기 스트리밍으로 클라우드 트래픽 최적화
- **상태 동기화:** `is_scanning`과 `current_pose` 트리 구조 체계적 기록

# 04 프로젝트 수행 경과 | 실시간 엔진 고도화 및 최적화 이력

메모리 효율화, 좌표 정밀도 개선, 트래픽 최적화 및 비동기 통신 아키텍처 도입

## 01. 데이터 관리 및 메모리 효율화

### ● 문제점: 메모리 과부하 및 병목

로컬 리스트 일괄 저장 방식으로 인한 메모리 점유율 수직 상승 및 업로드 시 네트워크 병목 발생

### ● 해결 방법: 순수 스트리밍 전환

캐싱 로직 제거 및 실시간 현재 좌표(current\_pose) 전송 구조 단순화를 통한 자원 최적화

## 02. 시각화 왜곡 및 물리 좌표 정밀도

### ● 문제점: 그래프 왜곡 및 오프셋 미반영

m 단위 사용 시 웹 시각화 비율 깨짐 발생, 툴 끝단(TCP)이 아닌 플랜지 중심 좌표 전송으로 인한 오차

### ● 해결 방법: 단위 변환 및 TCP 보정

m → mm 변환(x1000)으로 실제 비율 구현 및 TOOL\_LENGTH\_Z(0.2586) FK 연산 반영

## 03. 트래픽 최적화 및 전송 제어

### ● 문제점: 불필요한 상시 데이터 전송

비검사 구간에서의 지속적 데이터 송출로 인한 클라우드 비용 낭비 및 궤적 데이터 가독성 저하

### ● 해결 방법: 소프트웨어 게이트 도입

record\_enable 스위치 로직 추가 및 전송 주기를 0.033초(30Hz)로 최적화하여 트래픽 절감

## 04. 통신 지연 해결 (Worker Thread)

### ● 문제점: 동기식 전송에 의한 블로킹

Firebase 응답 대기 시 콜백 함수 정지로 로봇 제어 시스템 전체에 치명적인 Latency 유발

### ● 해결 방법: 비동기 워커 아키텍처

Queue & Threading 도입으로 제어 노드와 전송 스레드를 분리하여 통신 지연 영향도 0% 달성

# 04 프로젝트 수행 경과 | 웹 기반 디지털 트윈 3D 모니터링 시스템

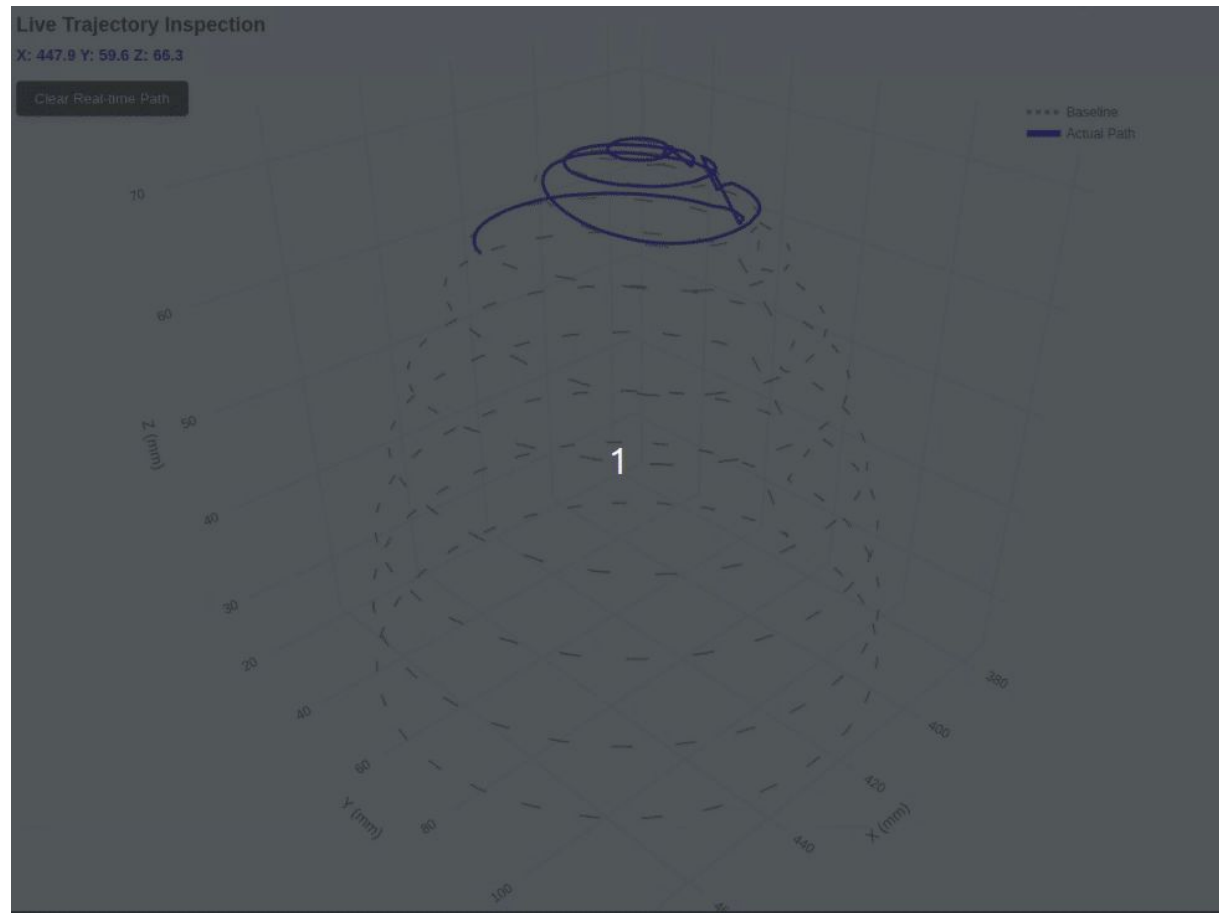
Plotly.js 및 Firebase Event-Driven 아키텍처를 활용한 무지연(Zero-latency) 궤적 시각화

## 핵심 렌더링 기술 (Core Rendering Tech)

- 고성능 부분 업데이트 (**extendTraces**): 매 프레임 전체를 다시 그리지 않고 수신 좌표만 이어 붙여 메모리 최소화 (최대 15,000pt 유지)
- 물리 공간 1:1 매칭 (**aspectmode: 'cube'**): X, Y, Z 축 비율을 고정하여 시각적 왜곡 방지 및 실제 로봇 움직임과 완벽한 비례 구현

## 이벤트 기반 동기화 및 세션 제어

- 비동기 실시간 리스너 (**onValue**): Firebase 웹소켓 기반 푸시로 기록 즉시 업데이트, 수동 새로고침 불필요
- 직관적 궤적 비교: Baseline(회색 점선) 대비 **Actual Path(파란 실선)**를 중첩하여 공정 이탈 즉시 판단
- Auto-Clear 메커니즘: is\_scanning: True** 감지 시 이전 궤적 자동 삭제 및 새 세션 시작



# 04 프로젝트 수행 경과 | 웹 기반 디지털 트윈 3D 모니터링 시스템

실시간 3D 디지털 트윈 시각화 기술의 단계별 문제점 및 해결 과정

## STEP 01. 초기 단계

### Three.js 기반 저수준 렌더링 (v00~v04)

#### ● 가시성 및 데이터 노이즈 문제

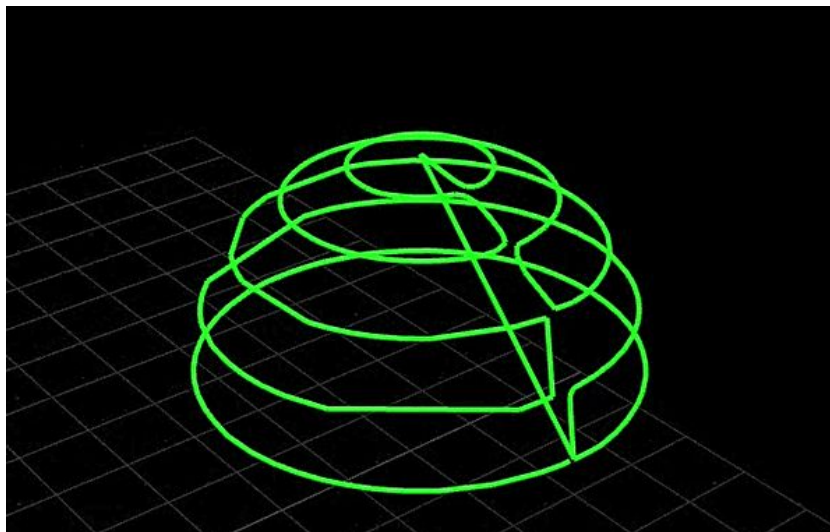
고정 뷰로 인한 관찰 제약 및 (0,0,0) 좌표 수신 시 궤적 왜곡 발생

#### ● OrbitControls & 원점 필터링 도입

자유로운 시점 전환 제어 및 유효 좌표 검증 로직으로 노이즈 원천 차단

#### ● 자동 줌 및 포인터 이벤트 최적화

BoundingBox 기반 로봇 추적 자동화 및 UI 레이어 분리로 간섭 해결



## STEP 02. 고도화 단계

### Plotly.js 기반 분석 엔진 전환 (v05~v09)

#### ● 대용량 데이터 렌더링 성능 저하

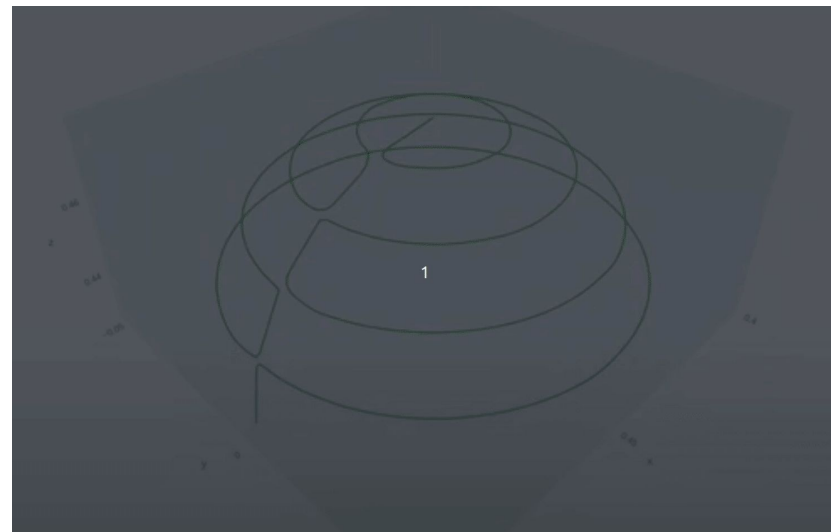
수만 개의 좌표를 매 프레임 다시 그릴 시 브라우저 부하 및 지연 발생

#### ● extendTraces 부분 업데이트 기법

전체 갱신 대신 데이터 이어 붙이기 방식으로 무지연 스트리밍 구현

#### ● 이중 트레이스 기반 실시간 비교 분석

Baseline(회색 점선) 대비 Actual(파란 실선) 중첩으로 공정 이탈 즉각 판단



# 04 프로젝트 수행 경과 | 실시간 데이터 수집 및 클라우드 파이프라인

ROS 2 기반 로봇 궤적 추적부터 Firebase 실시간 업로드까지의 아키텍처

## 01 수집 단계

### Node Start

baseline\_recorder

### Listen to ROS 2 Topics

- /dsr01/checking\_state (Bool)
- /dsr01/joint\_states (JointState)

### 상태 신호 제어 (Switch)

True: 스캔 시작 (RecOn)

False: 스캔 종료 (RecOff)

### 실시간 좌표 연산 (FK)

관절 각도(q) → 3D 좌표(X,Y,Z)

trajectory\_points 배열 누적

## 02 처리 단계

### Data Post-Processing

데이터 1/5 다운샘플링 압축

`pts = trajectory_points[::5]`

### Local Storage

로컬 PC baseline.npy 저장

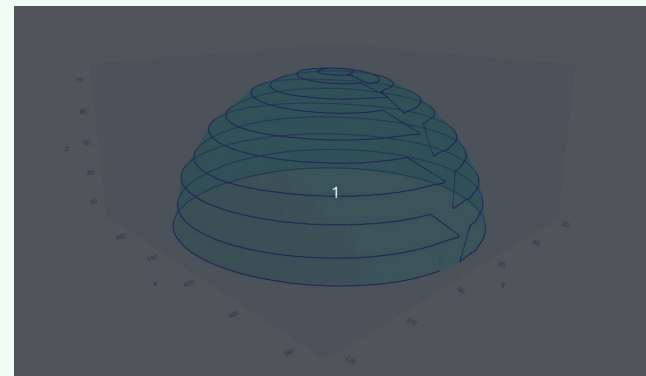
### Cloud Sync (Firebase RTDB)

좌표 배열 JSON 업로드

### HTML Gen (Plotly.js)

인터랙티브 리포트 시각화

## 03 완료 단계



### Storage & URL 발급

Firebase Storage 업로드

Public URL 생성 및 출력

✓ 다음 스캔 대기 모드

# 04 프로젝트 수행 경과 | 실시간 데이터 수집 및 클라우드 파이프라인

관절 데이터의 3D 공간 변환부터 지능형 시각화까지의 파이프라인

## 1 정기구학 (FK)

**역할:** TCP 3차원 좌표 산출

두산 M0609의 6개 관절 각도를 이용해 실제 툴 끝단의 X, Y, Z 위치를 정밀하게 계산합니다.

**방식:** DH Parameter 연산

4x4 동차 변환 행렬의 연속 곱셈을 수행합니다.

`T01 @ T12 @ ... @ T56`

## 2 상태 기반 콜백

**역할:** 선별적 데이터 기록

메인 프로그램의 "기록 시작" 신호에 동기화되어 필요한 작업 구간만 정밀 수집합니다.

**방식:** ROS 2 Topic 구독

/dsr01/checking\_state 토픽의 Boolean 값에 따라 리스트 적재 및 후처리를 제어합니다.

`True: Rec Start / False: Save`

## 3 데이터 경량화

**역할:** 전송 트래픽 최적화

클라우드 DB(Firebase) 전송 시 용량 초과와 통신 지연 문제를 사전에 방지합니다.

**방식:** Downsampling

파이썬 슬라이싱 기법으로 원본 데이터를 1/5 크기로 샘플링하여 효율을 극대화합니다.

`pts = trajectory_points[::5]`

## 4 Plotly 3D 렌더링

**역할:** 인터랙티브 시각화

수집된 원시 좌표를 사용자가 분석 가능한 3D 그래픽 리포트로 변환합니다.

**방식:** Scatter3d + Mesh3d

이동 선과 스캔 면적을 결합하여 웹에서 회전/확대 가능한 HTML 파일을 생성합니다.

[Interactive HTML Report](#)

# 04 프로젝트 수행 경과 | 실시간 데이터 수집 및 클라우드 파이프라인

모니터링 프레임워크의 단계별 문제점 및 해결 과정

## Phase 01

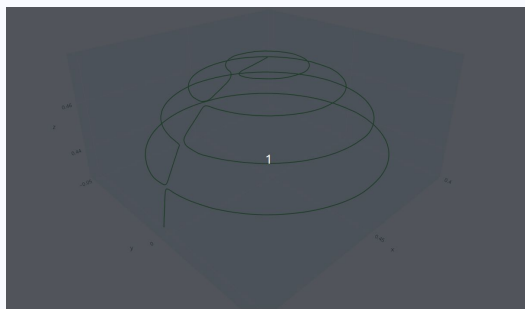
데이터 가시성 확보  
(00.py ~ 01.py)

- **문제:** 로컬 데이터 확인 불가. 수집 상태 확인을 위해 넘파이 배열 직접 열람 필요
- **해결:** **matplotlib** 도입. 3D 선 그래프 시각화 로직 추가로 데이터 형상 즉시 확인

## Phase 02

인터랙티브 리포트  
(02.py)

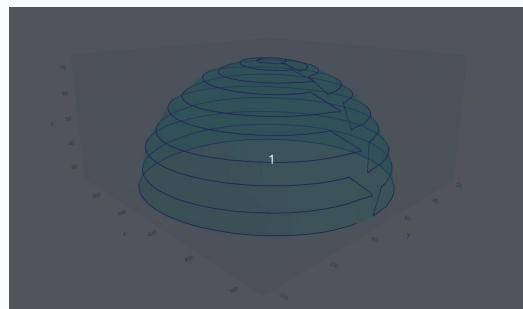
- **문제:** 정적 이미지(.png)의 한계. 3D 궤적의 측면/후면 오차 분석 불가
- **해결:** **Plotly** 프레임워크 교체. 회전/확대/축소가 가능한 HTML 리포트 생성



## Phase 03

실측 기반 3D 메쉬  
(03.py ~ 05.py)

- **문제:** 가상 모델과 실측 데이터의 괴리. 수학적 반구 모델의 기준점 부적합
- **해결:** **3D Mesh** 생성 로직 도입. 실제 주행 점들을 연결한 정밀 실측 모델 구축



## Phase 04

클라우드 및 최적화  
(06.py ~ 07.py)

- **문제:** 로컬 저장의 한계 및 데이터 방대함으로 인한 웹 로딩 성능 저하
- **해결:** Firebase 실시간 연동 및 **다운샘플링(1/5)** 도입. 통신 효율 및 렌더링 성능 극대화

## | 05 자체 평가 의견 | Firebase 클라우드 도입 성과



확장성 및 가용성

- ✓ 배포 단순화: 포트 개방이나 서버 유지보수 불필요
- ✓ 상태 지속성: 연결이 끊겨도 마지막 데이터 보존
- ✓ 자동 초기화: onDisconnect 기반 상태 동기화 최적화
- ✓ 보안 관리: Firebase 콘솔을 통한 손쉬운 인증

## | 05 자체 평가 의견 | 성능 병목: 폴링 기반 지연 분석



HTTP 폴링 방식은 실시간 조그(Jog) 버튼 조작 시 사용자가 체감하는 지연을 발생시킵니다.

## | 05 자체 평가 의견 | 향후 개선안: WebSocket 하이브리드



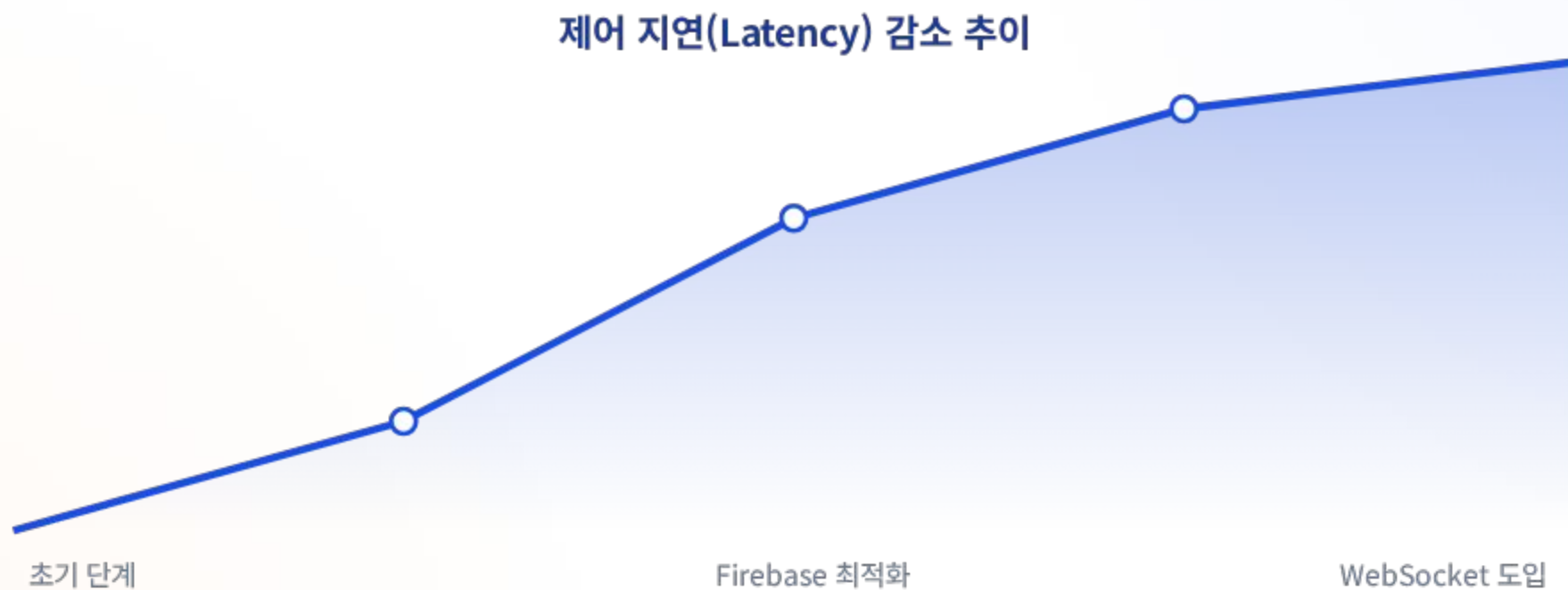
### 기술적 기대 효과

- 조그 반응성: 200ms → 10ms 이하 단축
- 안전성 강화: 비상정지 명령 즉각 전달 가능
- 트래픽 최적화: 불필요한 HTTP 요청 제거
- 부드러운 모니터링: 고주파 좌표 스트리밍 가능

## | 05 자체 평가 의견 | 시스템 구현 방식 비교 분석

| 평가 항목  | Firebase (현재)     | WebSocket (도입 예정)    |
|--------|-------------------|----------------------|
| 반응 속도  | 보통 (0.2s 폴링 지연)   | 매우 빠름 (실시간 스트리밍)     |
| 구현 난이도 | 낮음 (SDK 제공 기능 활용) | 높음 (커스텀 서버 및 재연결 구현) |
| 상태 유지력 | 매우 우수 (DB 영구 저장)  | 낮음 (연결 종료 시 데이터 휘발)  |
| 인프라 관리 | 불필요 (관리형 서비스)     | 필요 (ROS2 노드 내 서버 운영) |

## | 05 자체 평가 의견 | 성능 최적화 기대 결과



아키텍처 고도화를 통해 산업용 로봇 제어에 필수적인 실시간성을 확보할 수 있습니다.

# | 05 자체 평가 의견 | 코드 안정성 및 성능 개선 과제

## 복구 로직 개선

### 중복 호출 제거 및 메모리 참조

#### 문제점

조그 명령 시마다 좌표를 새로 읽어옴 (DSR\_ROBOT2)  
짧은 반복 입력 시 컨트롤러 부하 급증 및 지연 발생

#### 개선안

poll\_robot\_state에서 갱신된 최신 좌표를 메모리에서 즉시 참조  
응답 속도 향상 및 불필요한 통신 오버헤드 제거

## 스레드 안전성

### 명령 큐 기반 하이브리드 아키텍처

#### 일반 명령 (안전한 줄 서기)

[현재] 다중 스레드 혼용 (메인/인터럽트) → 충돌 위험 존재

- 조그, 이동 등 일반 명령은 명령 큐(Queue)에 적재
- 단일 전담 스레드(Worker)가 순차 처리하여 충돌 차단

#### 긴급 명령 (비상 새치기 - Bypass)

- 비상/일시정지는 대기열을 무시하고 즉시 호출
- Bypass를 통한 최우선 시스템 인터럽트 수행

# 감사합니다

Over-Crack: Doosan Robotics Crack Detection System

Q & A